

Java基础核心笔记

Java基础核心面试笔记

> 黑马程序员Java面试专题 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [面向对象基础](#1-面向对象基础)
2. [Java数据类型与运算](#2-java数据类型与运算)
3. [关键字详解](#3-关键字详解)
4. [异常体系](#4-异常体系)
5. [泛型详解](#5-泛型详解)
6. [String源码分析](#6-string源码分析)
7. [代码块执行顺序](#7-代码块执行顺序)
8. [深拷贝与浅拷贝](#8-深拷贝与浅拷贝)

1. 面向对象基础

1.1 封装、继承、多态 ★★★★★（高频）

封装：把属性私有化，提供公共访问方法

```
classDiagram
    class Person {
        -String name
        -int age
        +getName()
        +setName()
        +getAge()
        +setAge()
    }
```

继承：子类继承父类成员

```
classDiagram
    class Animal {
        +eat()
        +sleep()
    }

    class Dog {
        +eat()
        +bark()
    }

    class Cat {
        +eat()
        +meow()
    }
```

```
Animal <|-- Dog : extends
Animal <|-- Cat : extends
```

多态：同一引用调用同一方法，表现不同行为

```
flowchart LR
    A["A[\nAnimal animal]"] --> B["B{new Dog?}"]
    B --> C["C[Dog.eat]"]
    B --> D["D[new Cat]"]
    D --> E["E[Cat.eat]"]
```

多态的三个条件：

1. 继承关系

2. 方法重写

3. 父类引用指向子类对象

1.2 多态的成员访问规律 ★★★★★ (高频)

1.3 向上转型与向下转型 ★★★☆☆

```
flowchart TD
    A[Father] -->| | B[Son]
    B -->| | A

    F[Father f = new Son()] -->| | OK[ ]
    S[Father f2 = new Father()] -->| | CRASH[ClassCastException]
```

2. Java数据类型与运算

2.1 基本类型与引用类型 ★★★☆☆

```
flowchart LR
    subgraph
        B[byte] --> S[short] --> I[int] --> L[long]
        F[float] --> D[double]
        C[char] --> I
    end

    style B fill:#90EE90
    style L fill:#FFB6C1
```

自动类型提升规则：

1. 小类型向大类型提升 (byte→int→long→float→double)
2. byte、short、char参与运算自动提升为int
3. 多种类型混合运算，结果为最大操作数类型

2.2 装箱与拆箱 ★★★★★☆

```
//
Integer i = 10; // Integer.valueOf(10)

//
int num = i; // i.intValue()

// Integer-128~127
Integer a = 127;
Integer b = 127;
a == b; // true

Integer c = 128;
Integer d = 128;
c == d; // false
```

3. 关键字详解

3.1 static关键字 ★★★★★ (高频)

```
flowchart TD
    A[class] --> B[]
    B --> C[]
    C --> D[]

    subgraph
        E[]
        F[]
        G[]
    end
    end
```

特点：

- 类加载时执行，只执行一次

- 被所有对象共享
- 不能访问实例成员（没有this）
- 不能重写（是隐藏）

3.2 final关键字 ★★★★★（高频）

final变量初始化时机：

1. 定义时直接赋值
2. 构造方法中赋值
3. 静态代码块中赋值

3.3 abstract关键字 ★★★★★☆

3.4 interface接口 ★★★★★☆

```
classDiagram
    interface Flyable {
        <<interface>>
        +fly()
        +land()
    }

    interface Swimmable {
        <<interface>>
        +swim()
    }

    class Duck {
        +fly()
        +swim()
    }

    Flyable <|-- Duck : implements
    Swimmable <|-- Duck : implements
```

Java 8新增：

- default默认方法
- static静态方法

Java 9新增：

- private私有方法（辅助default）

3.5 super与this ★★★★★☆

> 规则：this()和super()不能同时出现，必须放在构造方法第一行。

3.6 volatile关键字 ★★★★★（高频）

```
flowchart LR
    subgraph A
        A1[volatile\nflag=true]
        A2[Store Barrier\n]
    end

    subgraph M
        M[flag=true]
    end

    subgraph B
        B1[Load Barrier\n]
        B2[flag\nflag=true]
    end

    A1 --> A2 --> M --> B1 --> B2
```

作用：保证可见性和有序性，不保证原子性。

3.7 synchronized关键字 ★★★★★（高频）

```
flowchart TD
    A[synchronized] --> B{}
    B --> C[this]
```

```

B -->|| D[class]
B -->|| E[]

F[] --> G[monitor]
G --> H[]
H --> I[monitor]

```

4. 异常体系

4.1 异常分类 ★★★☆☆

```

classDiagram
    class Throwable {
        +String message
        +getMessage()
        +printStackTrace()
    }

    class Error {
        +OutOfMemoryError
        +StackOverflowError
    }

    class Exception {
        +RuntimeException
        +IOException
    }

    class RuntimeException {
        +NullPointerException
        +ArithmeticException
        +ClassCastException
    }

    Throwable <|-- Error
    Throwable <|-- Exception
    Exception <|-- RuntimeException

```

4.2 Checked vs Unchecked ★★★★★☆

4.3 try-catch-finally执行顺序 ★★★★★ (高频)

```

flowchart TD
    A[try] --> B{return?}
    B -->|| C{finallyreturn?}
    B -->|| D[]
    C -->|| E[finallyreturn]
    C -->|| F[tryreturn\nfinally]

```

finally不执行情况：System.exit()终止JVM

5. 泛型详解

5.1 泛型擦除 ★★★★★ (高频)

```

flowchart LR
    A[GenericClass<T>] --> B[]
    B --> C[Object value]
    C --> D[T extends Number]
    D --> E[Number value]

```

规则：

- 无上限 → 擦除为Object
- 有上限 → 擦除为上限类型

5.2 PECS原则 ★★★★★☆

```

flowchart LR
    A[PECS] --> B[Producer Extends]
    A --> C[Consumer Super]

```

```
B --> D[extends\nList<? extends Number>]
C --> E[super\nList<? super Integer>]
```

> 记忆口诀：买外卖(Extends)→店家生产；卖废品(Super)→买家消费

5.3 泛型通配符对比

6. String源码分析

6.1 String不可变性 ★★★★★（高频）

```
public final class String {
    private final char[] value; //
}
```

不可变原因：

1. 字符串常量池需要
2. 安全性（防止被篡改）
3. 线程安全
4. hashCode缓存
5. 类加载器依赖

6.2 String Pool ★★★★★☆

```
flowchart LR
    subgraph
        A["String s1 = \"hello\""]
        B["String s2 = \"hello\""]
        A --> C[String Pool]
        B --> C
        A --> D["s1 == s2 true"]
    end

    subgraph new
        E["String s3 = new String(\"hello\")"]
        E --> F[]
        F --> G["s1 == s3 false"]
    end
```

6.3 String拼接 ★★★★★☆

```
//
String s1 = "a" + "b" + "c"; // = "abc"

// StringBuilder
String a = "a", b = "b";
String c = a + b + "c"; // new StringBuilder().append(a).append(b).append("c").toString()
```

7. 代码块执行顺序

7.1 代码块分类 ★★★★★☆

7.2 执行顺序 ★★★★★（高频）

```
flowchart TD
    A[new Son()] --> B[]
    B --> C[]
    C --> D[]
    D --> E[]
    E --> F[]
    F --> G[]
```

示例输出：

8. 深拷贝与浅拷贝

8.1 概念对比 ★★★☆☆

```
flowchart LR
    subgraph
        A1[] --> A2[]
        A1 -->|| A3[]
    end

    subgraph
        B1[] -->|| B2[]
        B2 -->|| B3[]
    end
end
```

8.2 实现方式

```
// Object.clone()
class Person implements Cloneable {
    int[] hobbies; //
    @Override
    protected Object clone() {
        return super.clone(); //
    }
}

// clone
class Person implements Cloneable {
    int[] hobbies;
    @Override
    protected Object clone() {
        Person p = (Person) super.clone();
        p.hobbies = this.hobbies.clone(); //
        return p;
    }
}
```

附录：面试高频问题速查

>

笔记说明：本笔记根据黑马程序员Java面试课程整理，涵盖Java基础核心面试知识点。使用mermaid图解便于理解，建议面试前快速回顾。

计算机网络核心笔记

计算机网络核心面试笔记

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [计算机网络体系结构](#1-计算机网络体系结构)
2. [TCP三次握手与四次挥手](#2-tcp三次握手与四次挥手)
3. [TCP可靠传输机制](#3-tcp可靠传输机制)
4. [TCP流量控制与拥塞控制](#4-tcp流量控制与拥塞控制)
5. [UDP协议](#5-udp协议)
6. [HTTP/HTTPS协议](#6-httphttps协议)
7. [DNS协议](#7-dns协议)
8. [Socket编程](#8-socket编程)

1. 计算机网络体系结构

1.1 五层协议体系 ★★☆☆☆

```
flowchart TB
    subgraph A [HTTP FTP DNS\nSMTP]
    end

    subgraph B [TCP UDP]
    end

    subgraph C [IP ICMP\nOSPF]
    end

    subgraph D [\nPPP]
    end

    subgraph E [\n]
    end

    A --> B --> C --> D --> E
```

1.2 各层数据单位 ★★☆☆☆

2. TCP三次握手与四次挥手

2.1 三次握手 ★★★★★ (高频)

```
sequenceDiagram
    participant C as Client
    participant S as Server

    Note over C: \nCLOSED
    Note over S: \nLISTEN

    C->>S: \nSYN=1, seq=x
    Note over C: SYN_SENT
    S->>C: \nSYN=1, ACK=1, seq=y, ack=x+1
    Note over S: SYN_RCVD
```

```

C->>S: \nACK=1, seq=x+1, ack=y+1
Note over C: ESTABLISHED
Note over S: ESTABLISHED

```

Note over C,S:

图解：

```

flowchart LR
    subgraph
        A1["A1[\nSYN=1, seq=x] -->|1| B1[]"]
        B1["B1 -->|2| A2[\nSYN=1, ACK=1, seq=y, ack=x+1]"]
        A2["A2 -->|3| B2[\nACK=1, seq=x+1, ack=y+1]"]
    end
end

```

为什么需要三次？

- 第一次：服务端确认客户端发送能力
- 第二次：客户端确认服务端发送+接收能力
- 第三次：服务端确认客户端接收能力

2.2 四次挥手 ★★★★★（高频）

```

sequenceDiagram
    participant C as Client
    participant S as Server

    Note over C,S: ESTABLISHED

    C->>S: \nFIN=1, seq=u
    Note over C: FIN_WAIT_1
    S->>C: \nACK=1, ack=u+1
    Note over C: FIN_WAIT_2
    Note over S: CLOSE_WAIT

    Note over S: ...

    S->>C: \nFIN=1, seq=w
    Note over S: LAST_ACK
    C->>S: \nACK=1, ack=w+1
    Note over C: TIME_WAIT
    Note over S: CLOSED

    Note over C: 2MSL\nTIME_WAIT->CLOSED

```

为什么需要四次？

- 第一次：客户端请求关闭
- 第二次：服务端确认，但可能还有数据要发送
- 第三次：服务端数据发送完毕，请求关闭
- 第四次：客户端确认，服务端关闭

2.3 状态转换 ★★★★★☆☆

```

stateDiagram-v2
    CLOSED --> LISTEN: listen
    CLOSED --> SYN_SENT: SYN

    LISTEN --> SYN_RCVD: SYN
    SYN_RCVD --> ESTABLISHED: ACK

    SYN_SENT --> ESTABLISHED: SYN+ACK
    SYN_SENT --> LISTEN: SYN

    ESTABLISHED --> FIN_WAIT_1: FIN
    FIN_WAIT_1 --> FIN_WAIT_2: ACK
    FIN_WAIT_2 --> TIME_WAIT: FIN

    CLOSE_WAIT --> LAST_ACK: FIN
    LAST_ACK --> CLOSED: ACK

    TIME_WAIT --> CLOSED: 2MSL

```

3. TCP可靠传输机制

3.1 可靠传输机制 ★★★★★☆

3.2 滑动窗口 ★★★★★☆

```
flowchart LR
    subgraph
        A[\n]
        B[\n]
        C[\n]
        D[]
    end

    subgraph
        E[\n]
        F[\n]
        G[]
    end
```

窗口大小：由接收方控制，用于流量控制

4. TCP流量控制与拥塞控制

4.1 流量控制 ★★★★★☆

```
flowchart LR
    A[] -->| | B[]
    B -->|ACK+| A
```

目的：防止发送方发送过快，导致接收方缓冲区溢出

4.2 拥塞控制 ★★★★★☆

```
flowchart TD
    A[] --> B["B{\nsssthresh?}"]
    B -->| | C[]
    B -->| | A
    C --> D["D{?}"]
    D -->| | E["E[sssthresh=sssthresh/2]"]
    D -->| | F[]
    C --> G["G{3\nACK?}"]
    G -->| | H["H[sssthresh=sssthresh/2]"]
    G -->| | I[]
```

拥塞控制算法：

1. **慢启动**：从小到大增加窗口
2. **拥塞避免**：线性增长
3. **快速重传**：收到3次重复ACK
4. **快速恢复**：快速恢复丢失报文段

5. UDP协议

5.1 UDP特点 ★★★★★☆

5.2 TCP vs UDP ★★★★★★（高频）

6. HTTP/HTTPS协议

6.1 HTTP请求方法 ★★★★★☆

6.2 HTTP状态码 ★★★★★☆

6.3 HTTPS原理 ★★★★★★（高频）

```

flowchart LR
    subgraph HTTPS
        A[] -->|1.ClientHello| B[]
        B -->|2.+| A
        A -->|3.+| B
        A -->|4.| B
    end
end

```

HTTPS建立连接过程：

1. ClientHello：客户端支持的TLS版本、加密算法、随机数C
2. ServerHello：服务端证书、公钥、随机数S
3. 客户端验证证书，用公钥加密预备主密钥发送给服务端
4. 双方用三个随机数生成对称密钥
5. 开始对称加密通信

7. DNS协议

7.1 DNS查询过程 ★★★☆☆

```

flowchart TD
    A[] --> B{?}
    B -->| | C[IP]
    B -->| | D[hosts]
    D --> E{?}
    E -->| | C
    E -->| | F[DNS]
    F --> G{?}
    G -->| | C
    G -->| | H[\\n-->]
    H --> C

```

7.2 DNS记录类型

8. Socket编程

8.1 TCP socket通信流程 ★★★☆☆

```

flowchart LR
    subgraph
        A[socket] --> B[bind]
        B --> C[listen]
        C --> D[accept]
        D --> E[read]
        E --> F[write]
    end
    end

    subgraph
        G[socket] --> H[connect]
        H --> I[write]
        I --> J[read]
    end
    end

    D -. ->| | H
    F -. ->| | H

```

8.2 一次HTTP请求的过程 ★★★★★（高频）

```

flowchart TD
    A[URL] --> B[DNS\\nIP]
    B --> C[TCP\\n]
    C --> D[HTTP]
    D --> E[]
    E --> F[HTTP]
    F --> G[HTML]
    G --> H[\\n]
    H --> I[]

```

附录：面试高频问题速查

> 笔记说明：本笔记结合小林coding和黑马程序员整理，涵盖计算机网络核心面试知识点。建议面试前快速回顾。

MySQL核心面试笔记

MySQL核心面试笔记

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [MySQL架构](#1-mysql架构)
2. [InnoDB行记录存储](#2-innodb行记录存储)
3. [索引详解](#3-索引详解)
4. [事务隔离级别与MVCC](#4-事务隔离级别与mvcc)
5. [MySQL锁机制](#5-mysql锁机制)
6. [日志系统](#6-日志系统)
7. [Buffer Pool](#7-buffer-pool)
8. [SQL优化](#8-sql优化)

1. MySQL架构

1.1 MySQL分层架构 ★★★☆☆

```
flowchart TB
    subgraph
        A[Connection]
        B[Pool]
    end

    subgraph
        C[SQL Interface]
        D[Parser]
        E[Optimizer]
        F[Cache]
    end

    subgraph
        G[InnoDB]
        H[MyISAM]
        I[Memory]
    end

    subgraph
        J[Physical Storage]
    end

    A --> C
    C --> D --> E
    E --> F
    F --> G
    G --> J
    H --> J
    I --> J
```

1.2 执行一条SQL的完整过程 ★★★★★

```
flowchart TD
    A[ ] --> B[ ]
    B --> C[ ]
    C --> D[ ]
    D --> E[ ]
    E --> F[InnoDB]
    F --> G[Redo Log/Binlog]
    G --> H[ ]
```

2. InnoDB行记录存储

2.1 行记录格式 ★★★☆☆

```
flowchart LR
    subgraph Compact
        A[] --> B[NULL]
        B --> C[]
        C --> D[1]
        D --> E[2]
        E --> F[...]
    end
    end
```

3. 索引详解

3.1 索引分类 ★★★★★☆

3.2 B+Tree vs B Tree vs 二叉树 ★★★★★ (高频)

B+Tree优势：

- 非叶子节点不存数据，同等高度存更多索引
- 叶子节点双向链表，适合范围查询
- 查询效率稳定（都在叶子节点）

3.3 聚簇索引 vs 二级索引 ★★★★★☆

```
flowchart LR
    subgraph
        A[B+Tree] --> B[]
    end
    end

    subgraph
        C[B+Tree] --> D[]
        D --> E[B+Tree]
        E --> F[]
    end
    end
```

回表查询：先查二级索引获取主键，再查主键索引获取完整数据

3.4 覆盖索引 ★★★★★☆☆

概念：查询的数据在二级索引叶子节点中就能找到，不需要回表

--

```
SELECT id, name FROM users WHERE name = '';
-- idnamename
```

3.5 索引失效场景 ★★★★★ (高频)

```
flowchart TD
    A[] --> B[1.\nlike '%xxx']
    A --> C[2.\nid + 1 = 10]
    A --> D[3.\nYEAR(date) = 2024]
    A --> E[4.\n]
    A --> F[5.0R\nname='xx' OR age=18]
    A --> G[6.\nABC,ABA]
    A --> H[7.!= / NOT IN\n]
```

最佳左前缀原则：对于联合索引(a,b,c)，查询必须使用a或a,b或a,b,c

4. 事务隔离级别与MVCC

4.1 四种隔离级别 ★★★★★☆

MySQL默认隔离级别：可重复读 (InnoDB)

4.2 并行事务问题 ★★★★★☆

```
flowchart LR
```

```

A[] -->|| B[A\n]
C[] -->|| D[A\nB]
E[] -->|| F[A5\nB1\nA6]

```

4.3 MVCC原理 ★★★★★ (高频)

```

flowchart TB
    subgraph MVCC
        A[] -->|trx_id| B[ID]
        A -->|roll_pointer| C[]
        C -->|| D[undo log\n]
    end

    subgraph
        E[] -->|roll_pointer| F[1]
        F -->|roll_pointer| G[2]
    end

    subgraph ReadView
        H[m_ids] -->|ID
        I[min_trx_id] -->|ID
        J[max_trx_id] -->|ID
    end

```

ReadView判断规则：

1. $trx_id < min_trx_id$: 可见
2. $trx_id > max_trx_id$: 不可见
3. trx_id in m_ids : 不可见，需顺着roll_pointer查找历史版本
4. trx_id not in m_ids : 可见

4.4 读已提交 vs 可重复读 ★★☆☆☆

4.5 幻读解决方案 ★★☆☆☆

5. MySQL锁机制

5.1 锁分类 ★★★★★☆

```

flowchart TB
    A[MySQL] --> B[]
    A --> C[]

    B --> B1[]
    B --> B2[]
    B --> B3[]

    C --> C1["C1[\nS]"]
    C --> C2["C2[\nX]"]

    B2 --> D["Record Lock\n"]
    B2 --> E["Gap Lock\n"]
    B2 --> F["Next-Key Lock\n"]

```

5.2 记录锁 vs 间隙锁 vs 临键锁 ★★★★★☆

5.3 死锁 ★★☆☆☆

```

flowchart LR
    A[A] -->|LOCK id=1| B[1]
    B -->|LOCK id=2| C[Bid=2]
    C -->|LOCK id=1| D[Aid=1]
    D -->|| A

```

死锁解决：

- 超时自动回滚 (innodb_lock_wait_timeout)
- 死锁检测，主动回滚小事务

6. 日志系统

6.1 三种日志对比 ★★★★★ (高频)

6.2 两阶段提交 ★★★★★☆

```
sequenceDiagram
    participant T as
    participant R as Redo Log Prepare
    participant B as Binlog
    participant D as

    T->>R: 1.Redo Log Prepare
    T->>B: 2.Binlog
    T->>R: 3.Redo Log Commit
    R->>D: 4.
```

7. Buffer Pool

7.1 Buffer Pool结构 ★★★★★☆

```
flowchart TB
    subgraph Buffer_Pool [Buffer Pool]
        A[Page 1] --> B[Page 2]
        B --> C[Page 3]
        C --> D[...]
    end

    subgraph
        E[]
        F[]
        G[]
        H[]
    end
    end
end
```

作用：缓存磁盘数据，减少磁盘IO

7.2 缓存淘汰策略 ★★★★★☆

LRU（最近最少使用）算法优化版：

- 预读机制
- 淘汰冷热数据（37分）

8. SQL优化

8.1 EXPLAIN分析 ★★★★★☆

```
EXPLAIN SELECT * FROM users WHERE name = '';
```

```
--
type: const/eq_ref/ref/range/all --
key: idx_name                      --
rows: 100                          --
Extra: Using index/Using where     --
```

8.2 慢查询优化 ★★★★★☆

8.3 分页查询优化 ★★★★★☆

```
--
SELECT * FROM orders ORDER BY id LIMIT 1000000, 10;

--
SELECT o.* FROM orders o
INNER JOIN (SELECT id FROM orders ORDER BY id LIMIT 1000000, 10) t
ON o.id = t.id;
```

附录：面试高频问题速查

>

笔记说明：本笔记结合小林coding图解MySQL和黑马程序员整理，涵盖MySQL核心面试知识点。建议面试前快速回顾。

Redis核心面试笔记

Redis核心面试笔记

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [Redis基础](#1-redis基础)
2. [Redis数据类型](#2-redis数据类型)
3. [Redis持久化机制](#3-redis持久化机制)
4. [Redis过期删除与内存淘汰](#4-redis过期删除与内存淘汰)
5. [Redis主从复制](#5-redis主从复制)
6. [Redis哨兵与Cluster](#6-redis哨兵与cluster)
7. [Redis缓存问题](#7-redis缓存问题)
8. [数据库与缓存一致性](#8-数据库与缓存一致性)

1. Redis基础

1.1 Redis vs Memcached ★★★★★☆☆

1.2 为什么用Redis作为缓存 ★★★★★☆☆

```
flowchart LR
    A[] -->| | B["B[\nQPS 10w+]"]
    C[] -->| | D["D[MySQL\nMySQL QPS 1w]"]
```

2. Redis数据类型

2.1 五种基本数据类型 ★★★★★★（高频）

2.2 数据类型底层实现 ★★★★★☆☆

```
flowchart TB
    A[String] --> B["SDS\n"]
    C[List] --> D["quicklist\n"]
    E[Hash] --> F["\n"]
    G[Set] --> H["\n"]
    I[Zset] --> J["\n"]
```

2.3 SDS vs C字符串 ★★★★★☆☆

2.4 跳表原理 ★★★★★☆☆

```
flowchart TB
    subgraph subgraph
        A["Level 3"] -->| | D["Node 10"]
        A --> B["Level 2"] --> C["Level 1"]
        B --> D
        C --> E["Node 5"]
        C --> F["Node 15"]
        D --> F
    end
```

特点：多层索引，快速定位， $O(\log N)$ 查找

3. Redis持久化机制

3.1 RDB vs AOF对比 ★★★★★☆

3.2 RDB持久化 ★★★★★☆

```
flowchart LR
    A[RDB] -->|SAVE| B[]
    A -->|BGSAVE| C[fork]
```

3.3 AOF持久化 ★★★★★☆

```
flowchart TD
    A[AOF] --> B[always\n]
    A --> C[everysec\n]
    A --> D[no\n0S]

    B -->| | E[]
    C --> F[\n]
    D --> G[\n]
```

AOF重写：BGSAVE重写AOF，合并重复命令，减小文件大小

4. Redis过期删除与内存淘汰

4.1 过期删除策略 ★★★★★☆

Redis采用：惰性删除 + 定期删除

4.2 内存淘汰策略 ★★★★★☆

```
flowchart LR
    A[] --> B[noeviction\n]
    A --> C[volatile\nkey]
    A --> D[allkeys\nkey]

    C --> E[LRU/LFU/RANDOM]
    D --> F[LRU/LFU/RANDOM]
```

5. Redis主从复制

5.1 主从复制原理 ★★★★★☆

```
flowchart TD
    A[master] -->|SYNC/PSYNC| B[slave]
    A -->|RDB| C[]
    C --> D[]

    A -->| | E[]
    E --> F[]
```

5.2 全量同步 vs 增量同步 ★★★★★☆

6. Redis哨兵与Cluster

6.1 哨兵模式 ★★★★★☆

```
flowchart TB
    subgraph
        A[Sentinel 1]
        B[Sentinel 2]
        C[Sentinel 3]
    end

    subgraph
        D[]
        E[1]
        F[2]
    end

    end
```

```

A --> D
B --> D
C --> D
D --> E
D --> F

```

作用：监控、自动故障转移、通知

6.2 Redis Cluster ★★★☆☆

```

flowchart TB
    subgraph Cluster
        A[Node 1] --> B[Node 2]
        B --> C[Node 3]
        C --> A

        D[Node 4] --> E[Node 5]
        E --> F[Node 6]
        F --> D
    end
end

```

特点：16384个槽位，自动分片，高可用

7. Redis缓存问题

7.1 缓存雪崩 ★★★☆☆

```

flowchart LR
    A[key] --> B[]
    B --> C[]
    C --> D[]

```

解决方案：

- 过期时间加上随机值
- 互斥锁重建
- 热点数据永不过期

7.2 缓存击穿 ★★★☆☆

```

flowchart LR
    A[key] --> B[]
    B --> C[]
    C --> D[]

```

解决方案：

- 互斥锁
- 热点数据永不过期
- 逻辑过期

7.3 缓存穿透 ★★★☆☆

```

flowchart LR
    A[\n] --> B[]
    B --> C[]
    C --> D[]

```

解决方案：

- 布隆过滤器
- 空对象缓存
- 参数校验

8. 数据库与缓存一致性

8.1 常见方案 ★★★☆☆

```

flowchart LR
    A[Cache Aside] --> B[\n]

```

```
C[Read Through] -->|\n]
D[Write Through] -->|\n]
E[Write Behind] -->|\n]
```

8.2 Cache Aside模式 ★★★★★

```
sequenceDiagram
    participant A as
    participant C as
    participant D as
```

Note over A,C,D:

A->>C: 1.

C->>A: 2.

C->>D: 3.

D-->>C: 4.

C-->>A: 5.

Note over A,C,D:

A->>D: 1.

D-->>A: 2.

A->>C: 3.

附录：面试高频问题速查

>

笔记说明：本笔记结合小林coding图解Redis和黑马程序员整理，涵盖Redis核心面试知识点。建议面试前快速回顾。

Spring全家桶核心笔记

Spring全家桶核心面试笔记

> 黑马程序员Java面试专题 + 小林coding面试题整理 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [Spring核心IoC](#1-spring核心ioc)
2. [Spring AOP](#2-spring-aop)
3. [Spring事务](#3-spring事务)
4. [Spring循环依赖](#4-spring循环依赖)
5. [SpringMVC](#5-springmvc)
6. [SpringBoot自动装配](#6-springboot自动装配)
7. [SpringCloud核心组件](#7-springcloud核心组件)
8. [SpringBeanFactory与FactoryBean](#8-springbeanfactory与factorybean)
9. [高频面试题补充](#9-高频面试题补充)

1. Spring核心IoC

1.1 IoC控制反转 ★★★★★☆

```
flowchart LR
    subgraph
        A[UserService] -->|new| B[UserDao]
    end

    subgraph IoC
        C[Spring] -->| | D[UserService]
        D -->| | E[UserDao]
    end
```

IoC好处：

- 降低对象耦合
- 便于单元测试
- 提高可维护性

1.2 BeanFactory vs ApplicationContext ★★★★★☆☆

1.3 Bean生命周期 ★★★★★★（高频）

```
flowchart TD
    A[1.\n] --> B[2.\n@Autowired]
    B --> C[3.]
    C --> C1[BeanNameAware]
    C1 --> C2[BeanFactoryAware]
    C2 --> C3[ApplicationContextAware]
    C3 --> C4[BeanPostProcessor\n]
    C4 --> C5[@PostConstruct]
    C5 --> C6[InitializingBean\nafterPropertiesSet]
    C6 --> C7[init-method]
    C7 --> C8[BeanPostProcessor\n]
    C8 --> D[4.]
    D --> E[5.]
    E --> E1[@PreDestroy]
    E1 --> E2[DisposableBean\nndestroy]
    E2 --> E3[destroy-method]
```

1.4 依赖注入方式 ★★★★★☆

1.5 @Autowired vs @Resource vs @Inject ★★★★★☆

2. Spring AOP

2.1 AOP核心概念 ★★★★★☆

flowchart TB

```
A[Aspect] --> B[JoinPoint]
B --> C[Pointcut]
C --> D[Advice]
D --> E[Target]
E --> F[Weaving]
```

五类通知：

2.2 通知执行顺序 ★★★★★ (高频)

flowchart TD

```
A[@Around] --> B[@Before]
B --> C[]
C --> D{?}
D -->|| E[@AfterReturning]
D -->|| F[@AfterThrowing]
E --> G[@After]
F --> G
G --> H[@Around]
```

2.3 Spring AOP vs AspectJ

2.4 JDK动态代理 vs CGLIB代理 ★★★★★ (高频)

Spring选择规则：

- 目标类实现了接口：默认使用JDK动态代理
- 目标类未实现接口：使用CGLIB代理
- 可以通过`proxy-target-class="true"`强制使用CGLIB

3. Spring事务

3.1 事务隔离级别 ★★★★★ (高频)

MySQL默认：REPEATABLE_READ

Oracle默认：READ_COMMITTED

3.2 事务传播行为 ★★★★★ (高频)

NESTED vs REQUIRES_NEW：

- REQUIRES_NEW：完全独立的事务，内外事务回滚互不影响
- NESTED：嵌套事务，外事务回滚会导致内事务回滚，但内事务回滚不会影响外事务

3.3 @Transactional失效场景 ★★★★★ (高频)

flowchart TD

```
A[] --> B[public]
A --> C[this]
A --> D[catch]
A --> E[]
A --> F[]
A --> G[]

B -->|| B1[publicAspectJ]
C -->|| C1[]
D -->|| D1[]
E -->|| E1[rollbackFor=Exception.class]
```

详细失效场景：

1. **非public方法**：`@Transactional`只对public方法生效
2. **自调用**：`this.xxx()`绕过了代理，直接调用同类其他@Transactional方法

3. ****异常被吞****：catch块中没有重新抛出异常
4. ****异常类型不匹配****：默认只对RuntimeException回滚，需指定rollbackFor
5. ****同类内部调用****：同类中一个方法调用另一个@Transactional方法
6. ****传播行为不当****：NESTED在某些数据库不支持

解决自调用问题：

```
// 1
@Autowired
private MyService self;

// 2
((MyService)AopContext.currentProxy()).xxx();

---
```

4. Spring循环依赖

4.1 什么是循环依赖 ★★★★★☆

```
flowchart LR
    A[ServiceA] -->| | B[ServiceB]
    B -->| | A
```

```
@Service
public class A {
    @Autowired private B b;
}
```

```
@Service
public class B {
    @Autowired private A a;
}
```

三种循环依赖场景：

4.2 三级缓存解决循环依赖 ★★★★★★（高频）

```
flowchart TB
    subgraph singletonObjects
        A1[Bean]
    end

    subgraph earlySingletonObjects
        A2[Bean\n]
    end

    subgraph singletonFactories
        A3[ObjectFactory\nlambda]
    end

    A3 -->| | A2
    A2 -->| | A1
```

三级缓存详解：

解决流程（AB循环依赖）：

1. 创建A，实例化后暴露ObjectFactory到三级缓存
2. 注入B时发现需要B，开始创建B
3. B实例化后暴露ObjectFactory到三级缓存
4. B注入A时，从三级缓存获取A的ObjectFactory
5. 调用ObjectFactory.getObject()获取A的早期引用
6. 将早期引用存入二级缓存，删除三级缓存中的A
7. B创建完成，存入一级缓存
8. A继续创建，获取到B，完成注入和初始化
9. A存入一级缓存，删除二级缓存中的A

4.3 为什么需要三级缓存？ ★★★★★☆☆

只用二级缓存不行吗？

答案是不行。核心原因是为了正确处理AOP代理。

场景分析：假设A需要被代理（加了@Transactional）

1. 如果只有二级缓存：
 - B创建时从二级缓存获取A的早期引用
 - 这个引用是A的原始对象（非代理）
 - A初始化完成后生成代理对象
 - 结果：B持有的A是原始对象，而容器中的A是代理对象
 - ****单例对象不一致！****
2. 三级缓存的作用：
 - ObjectFactory.getObject()会判断是否需要生成代理
 - 如果需要，提前生成代理对象放入二级缓存
 - 这样B获取到的就是代理对象
 - A初始化完成后直接使用代理对象，保证单例一致性

三级缓存的本质：延迟代理对象的生成，保证循环依赖时获取到的是最终形态的对象。

4.4 循环依赖解决方案深入 ★★★★★（高频）

构造器循环依赖为什么不能解决？

构造器注入需要在构造函数中获取依赖对象，而此时对象尚未创建完成，无法暴露早期引用，导致死循环。

prototype作用域循环依赖为什么不能解决？

prototypeBean每次获取都会创建新实例，Spring不会缓存这类Bean，因此无法提前暴露引用。

5. SpringMVC

5.1 DispatcherServlet处理流程 ★★★★★（高频）

flowchart TD

```
A[HTTP] --> B[DispatcherServlet]
B --> C[HandlerMapping\nHandler]
C --> D[HandlerAdapter\nHandler]
D --> E[ModelAndView]
E --> F[ViewResolver\n]
F --> G[View]
G --> H[HTTP]
```

九大组件：

6. SpringBoot自动装配

6.1 @SpringBootApplication注解 ★★★★★☆

@SpringBootApplication

```
├─ @Configuration      //
├─ @EnableAutoConfiguration  //
└─ @ComponentScan      //
```

6.2 自动装配原理 ★★★★★（高频）

flowchart TD

```
A[@SpringBootApplication] --> B[@EnableAutoConfiguration]
B --> C[SpringFactoriesLoader]
C --> D[META-INF/spring.factories]
D --> E[EnableAutoConfiguration]
E --> F[@Conditional]
F --> G[@Bean]
```

自动装配流程详解：

1. ****@EnableAutoConfiguration****触发自动装配
2. ****SpringFactoriesLoader****从`META-INF/spring.factories`加载配置
3. 读取所有`EnableAutoConfiguration`对应的类

4. 通过`@Conditional`条件筛选符合条件的配置类

5. 将符合条件的`@Bean`方法注册到容器

spring.factories示例：

```
# org.springframework.boot.autoconfigure.EnableAutoConfiguration
org.springframework.boot.autoconfigure.AutoConfigureOrder=\
org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
org.springframework.boot.autoconfigure.web.servlet.WebMvcAutoConfiguration
```

6.3 @Conditional条件注解 ★★★★★☆

6.4 Starter原理 ★★★★★☆

flowchart LR

```
A[xxx-spring-boot-starter] --> B[]
B --> C[xxx-starter-autoconfigure]
C --> D[]
D --> E[Bean]
```

Starter标准结构：

```
xxx-spring-boot-starter/
├ pom.xml
└ src/main/resources/
    └ META-INF/
        └ spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

7. SpringCloud核心组件

7.1 微服务架构图 ★★★★★☆

flowchart TB

```
A[ Gateway] --> B[]
B --> C[Nacos\n]
B --> D[Feign\n]
D --> E[1]
D --> F[2]
```

```
G[Sentinel\n] --> D
H[Config\n] --> B
```

7.2 Eureka vs Nacos CAP对比 ★★★★★☆

7.3 Feign原理 ★★★★★☆

flowchart LR

```
A[] --> B[@FeignClient]
B --> C[]
C --> D[]
D --> E[HTTP]
E --> F[Ribbon]
F --> G[]
```

7.4 Sentinel vs Hystrix

8. SpringBeanFactory与FactoryBean

8.1 概念区分 ★★★★★★（高频面试题）

8.2 BeanFactory接口 ★★★★★☆

```
public interface BeanFactory {
    // Bean
    Object getBean(String name);

    // Bean
    <T> T getBean(String name, Class<T> requiredType);

    //
}
```

```

    boolean containsBean(String name);

    // Bean
    Class<?> getType(String name);

    //
    boolean isSingleton(String name);
}

```

常见实现类：

- `DefaultListableBeanFactory`：最常用的实现
- `XmlBeanFactory`：已废弃，从XML加载
- `ApplicationContext`：BeanFactory的子接口，提供更多企业级功能

8.3 FactoryBean接口 ★★★★★（高频面试题）

```

public interface FactoryBean<T> {
    //
    T getObject();

    //
    Class<?> getObjectType();

    //
    default boolean isSingleton() {
        return true;
    }
}

```

FactoryBean vs BeanFactory：

```

BeanFactory IOC    →  getBean() Bean
FactoryBeanBean   →  getBean() getObject()

```

FactoryBean使用场景：

1. **第三方框架集成**：MyBatis的`SqlSessionFactoryBean`、Shiro的`RealmFactoryBean`
2. **定制Bean创建逻辑**：创建复杂对象、代理对象
3. **延迟加载**：延缓Bean的创建时机

8.4 FactoryBean实现示例 ★★★★★☆

```

// FactoryBean
@Component
public class MyFactoryBean implements FactoryBean<User> {

    @Override
    public User getObject() {
        //
        User user = new User();
        user.setName("factoryCreated");
        return user;
    }

    @Override
    public Class<?> getObjectType() {
        return User.class;
    }

    @Override
    public boolean isSingleton() {
        return true;
    }
}

//
@Autowired
private BeanFactory beanFactory;

// FactoryBean&
MyFactoryBean factoryBean = (MyFactoryBean) beanFactory.getBean("&myFactoryBean");

// FactoryBeanBean
User user = (User) beanFactory.getBean("myFactoryBean");

```

8.5 Spring内部常用FactoryBean ★★★☆☆

9. 高频面试题补充

9.1 Spring框架设计模式 ★★★☆☆

9.2 Spring是单例还是多例？线程安全问题 ★★★★★☆

默认单例：Spring容器中的Bean默认是单例的。

线程安全问题：

- 单例Bean如果存在可变共享状态（如实例变量），多线程环境下会存在线程安全问题
- 解决方案：
 1. 将Bean设置为prototype（每次创建新实例）
 2. 使用ThreadLocal存储线程本地变量
 3. 保证Bean无状态（不存储可变状态）

9.3 Spring启动流程 ★★★☆☆

flowchart TD

```
A[ApplicationContext] --> B[BeanDefinition]
B --> C[BeanFactoryPostProcessor]
C --> D[BeanPostProcessor]
D --> E[Bean]
E --> F[]
F --> G[]
```

9.4 Spring如何处理依赖冲突 ★★★☆☆

9.5 @ComponentScan扫描规则 ★★★☆☆

```
@ComponentScan(
    basePackages = {"com.example"}, //
    includeFilters = {}, //
    excludeFilters = {}, //
    useDefaultFilters = true //
)
```

默认扫描：@Component、@Repository、@Service、@Controller、@Configuration

附录：面试高频问题速查

小林coding面试高频追问

Q1：Spring为什么用三级缓存而不是二级？

参考答案：

核心是为了正确处理AOP代理。如果只用二级缓存，当B依赖A时，B获取到的是A的原始对象。但A在初始化完成后才会生成代理对象，导致B持有的A和容器中的A不一致，违反单例原则。

三级缓存的ObjectFactory会在getObject()时判断是否需要生成代理，如果需要则提前生成代理放入二级缓存，保证循环依赖时获取的是最终形态。

Q2：@Transactional的rollbackFor为什么不指定Throwable？

参考答案：

指定Throwable会导致所有异常都回滚，包括IOException等checked异常。但实际业务中很多checked异常是需要捕获处理的，不应该直接回滚。默认的RuntimeException回滚策略更符合实际业务需求。

Q3：构造器注入为什么推荐？

参考答案：

1. 保证依赖不为null，编译期就能发现问题
2. 依赖不可变（final字段）
3. 测试友好，无需Spring容器也能测试
4. 避免循环依赖问题（构造器循环依赖本身就不能解决）

Q4：SpringBoot和SpringMVC什么关系？

参考答案：

- SpringMVC是Web框架，处理Web请求
- SpringBoot是自动配置工具，简化Spring配置
- SpringBoot自动配置了SpringMVC（WebMvcAutoConfiguration）
- 关系：SpringBoot SpringMVC（Boot自动配置了MVC）

>

笔记说明：本笔记根据黑马程序员Spring面试课程和小林coding面试题整理，涵盖Spring全家桶核心面试知识点。使用mermaid图解便于理解，建议面试前快速回顾。

Java集合框架源码笔记

Java集合框架源码分析面试笔记

> 黑马程序员Java面试专题 + 小林coding面试题整理 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [集合框架整体结构](#1-集合框架整体结构)
2. [ArrayList源码分析](#2-arraylist源码分析)
3. [LinkedList源码分析](#3-linkedlist源码分析)
4. [HashMap源码分析](#4-hashmap源码分析)
5. [ConcurrentHashMap源码分析](#5-concurrenthashmap源码分析)
6. [Hashtable vs HashMap vs ConcurrentHashMap对比](#6-hashtable-vs-hashmap-vs-concurrenthashmap对比)
7. [TreeMap源码分析](#7-treemap源码分析)
8. [LinkedHashMap原理](#8-linkedhashmap原理)
9. [Queue体系结构](#9-queue体系结构)
10. [各集合线程安全性对比](#10-各集合线程安全性对比)
11. [fail-fast vs fail-safe](#11-fail-fast-vs-fail-safe)
12. [各集合适用场景](#12-各集合适用场景)

1. 集合框架整体结构

1.1 两大体系 ★★★★★☆☆

```
flowchart TB
    subgraph Collection
        A[List]
        B[Set]
        C[Queue]
    end

    subgraph Map
        D[HashMap]
        E[TreeMap]
        F[LinkedHashMap]
    end

    Iterable <|-- Collection
    Collection <|-- List
    Collection <|-- Set
    Map <|-- HashMap
    Map <|-- TreeMap
```

1.2 List vs Set vs Map

1.3 线程安全分类 ★★★★★☆☆

2. ArrayList源码分析

2.1 核心属性 ★★★★★☆☆

```
public class ArrayList<E> {
    //
    private static final int DEFAULT_CAPACITY = 10;
    //
}
```

```

private static final Object[] EMPTY_ELEMENTDATA = {};

//
transient Object[] elementData;

//
private int size;
}

```

2.2 扩容机制 ★★★★★ (高频)

```

flowchart TD
    A[add] --> B{?}
    B --> C[]
    B --> D[grow]
    D --> E["E = * 1.5 + 1"]
    E --> F[]
    F --> G["G[Arrays.copyOf]"]
    G --> C

```

扩容公式： $\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} \gg 1)$ 即 1.5倍

2.3 线程安全问题 ★★★★★ (高频面试题)

ArrayList不是线程安全的，多线程并发修改会暴露以下问题：

解决方案：

```

// 1Collections.synchronizedList
List<Object> list = Collections.synchronizedList(new ArrayList<>());

// 2CopyOnWriteArrayList
CopyOnWriteArrayList<Object> list = new CopyOnWriteArrayList<>();

// 3Vector
Vector<Object> list = new Vector<>();

```

2.4 fail-fast机制 ★★★★★☆

```

flowchart LR
    A[] --> B["B[expectedModCount\n= modCount]"]
    B --> C[next]
    C --> D["D{modCount ==\nexpectedModCount?}"]
    D --> E[]
    D --> F["F[\nConcurrentModificationException]"]

```

原因：modCount记录集合结构性修改次数，迭代器检测到变化则快速失败。

3. LinkedList源码分析

3.1 双向链表结构 ★★★★★☆☆

```

flowchart TB
    subgraph LinkedList
        A[first] --> B[Node1]
        B --> C[Node2]
        C --> D[Node3]
        D --> E[last]

        B ---|prev| A
        C ---|prev| B
        D ---|prev| C
        E ---|prev| D
    end

```

Node结构：

```

private static class Node<E> {
    E item;           //
    Node<E> next;      //
    Node<E> prev;      //
}

```

3.2 时间复杂度对比

4. HashMap源码分析

4.1 JDK7 vs JDK8结构变化 ★★★★★（高频）

```
flowchart LR
    subgraph JDK7
        A1[+\n]
    end

    subgraph JDK8
        A2[+/\n]
    end
```

红黑树转换条件（同时满足）：

1. 链表长度 > 8
2. 数组容量 >= 64（否则先扩容）

为什么用红黑树而不是平衡二叉树？

HashMap要求的是综合增删改查性能，AVL树插入时要频繁调整影响性能，红黑树牺牲部分查找性能但大大减少调整次数。

4.2 哈希算法（扰动函数） ★★★★★

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

```
flowchart LR
    A[key.hashCode] --> B[h = hashCode]
    B --> C[h ^ (h >>> 16)]
    C --> D[\n]
    D --> E[\nhash & (length-1)]
```

4.3 扩容机制 ★★★★★（高频）

```
flowchart TD
    A[ > *] --> B[\n]
    B --> C[]
    C --> D{hash & oldCap}
    D -->|==0| E[]
    D -->|!=0| F[ + oldCap]
```

判断原理：

- oldCap=16 (10000)
- hash1 & 16 = 0 → 原位置
- hash2 & 16 = 16 → 原位置+16

扩容优化：不需要重新计算hash，只需要看hash值的最高位是0还是1

4.4 HashMap为什么线程不安全 ★★★★★（高频面试题）

JDK1.7头插法问题：

- 扩容时链表会反转，多线程环境下形成环形链表
- 导致死循环、CPU 100%

JDK1.8尾插法问题：

- 数据覆盖：多线程put时，计算出的索引相同，后一个覆盖前一个
- size不准确：size++不是原子操作
- 哈希碰撞攻击：精心构造的hash值使链表过长

具体问题场景：

```
// 1
// AB5put

// 2size
// size++1
```

```
// 3JDK1.7
// ++ = +
```

4.5 HashMap大小为什么是2的n次方 ★★★★★☆

核心原因：

1. **位运算替代取模**：`hash & (length-1)` 等价于 `hash % length`
2. **减少哈希碰撞**：length-1 二进制全1，能充分利用hash值的分布
3. **扩容优化**：只需要判断hash & oldCap，不需要重新计算hash

反例：如果length=15，length-1=14(1110)，最后一位永远是0，浪费一半空间

5. ConcurrentHashMap源码分析

5.1 JDK7 vs JDK8对比 ★★★★★ (高频)

```
flowchart LR
    subgraph JDK7
        A[Segment[0]] --> A1[HashEntry]
        A[Segment[1]] --> A2[HashEntry]
        A[Segment[n]] --> A3[HashEntry]
    end

    subgraph JDK8CAS+synchronized
        B[Node] --> B1[/]
        B --> B2[/]
        B --> B3[/]
    end
```

5.2 分段锁 vs CAS+synchronized ★★★★★ (高频面试题)

JDK7分段锁原理：

```
//
Segment<K,V>[] segments;

// SegmentHashMap
static class Segment<K,V> extends ReentrantLock {
    transient volatile HashEntry<K,V>[] table;
}
```

优点：

- 并发度高，不同Segment之间互不影响
- 16个Segment理论上最多16线程并发

缺点：

- 锁粒度还是比较粗（一个Segment一把锁）
- 初始化时需要创建所有Segment

5.3 JDK8 CAS+synchronized详解 ★★★★★ (高频)

```
flowchart TD
    A[put] --> B{?}
    B --> C[CAS]
    C --> D[]
    B --> D
    D --> E{?}
    E --> F[CAS]
    E --> G[synchronized]
    G --> H[key?]
    H --> I[value]
    H --> J[]
    J --> K[]
```

CAS使用场景：

- 数组初始化
- 桶为空时的插入

synchronized使用场景：

- 桶不为空（发生hash碰撞）
- 需要遍历链表/红黑树

5.4 为什么JDK8改用synchronized？ ★★★☆☆

JDK8优势：

- 锁粒度更细，并发度更高
- JVM对synchronized持续优化（偏向锁、轻量级锁）
- 红黑树优化查询性能

6. Hashtable vs HashMap vs ConcurrentHashMap对比

6.1 核心对比 ★★★★★（高频）

6.2 选型建议

```
flowchart TD
    A[Map?] --> B{?}
    B --> C{?}
    B --> D[HashMap]
    C --> E[ConcurrentHashMap]
    C --> F[Collections.synchronizedMap]
```

7. TreeMap源码分析

7.1 红黑树五大性质 ★★★☆☆

```
flowchart TB
    A[] --> B[1. ]
    A --> C[2. ]
    A --> D[3. ]
    A --> E[4. ]
    A --> F[5. \n]
```

7.2 插入调整三种情况 ★★★☆☆

8. LinkedHashMap原理

8.1 结构特点 ★★★☆☆

```
flowchart TB
    subgraph LinkedHashMap
        A[HashMap] --> B[]
        B --> C[\n]
    end
```

8.2 LRU缓存实现 ★★★★★（高频）

```
// accessOrder=true
public class LRUCache<K, V> extends LinkedHashMap<K, V> {

    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder=true
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}
```

9. Queue体系结构

9.1 Queue vs Deque ★★★☆☆

```
flowchart TB
    subgraph Queue
        A[add/offer] --> B[remove/poll]
        B --> C[element/peek]
    end

    subgraph Deque
        D[addFirst/offerFirst] --> E[addLast/offerLast]
        E --> F[removeFirst/pollFirst]
        F --> G[removeLast/pollLast]
    end
```

9.2 阻塞队列分类

10. 各集合线程安全性对比

10.1 线程安全类总览 ★★★★★☆

11. fail-fast vs fail-safe

11.1 概念对比 ★★★★★★（高频面试题）

```
flowchart LR
    subgraph fail-fast
        A[modCount] --> B[]
    end

    subgraph fail-safe
        C[] --> D[]
    end
```

11.2 fail-fast触发场景 ★★★★★☆

```
// 1
List<String> list = new ArrayList<>();
list.add("a");
list.add("b");
for (String s : list) { // ConcurrentModificationException
    list.remove(s);
}

// 2
// ABmodCount
```

11.3 fail-safe解决方案 ★★★★★☆

```
// CopyOnWriteArrayList
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
//
//

//
Iterator<String> iter = list.iterator();
//
```

11.4 ConcurrentHashMap的弱一致性 ★★★☆☆

弱一致性体现：

- 遍历时可能看到部分修改
- size()可能不准确
- isEmpty()可能不准确

原因：为了提高并发性能，不在读操作加锁

与fail-safe的区别：

- fail-safe：复制副本遍历
- ConcurrentHashMap：不复制，但部分操作无锁

12. 各集合适用场景

12.1 List选择 ★★★★★☆

12.2 Map选择 ★★★★★★（高频面试题）

flowchart TD

```
A[Map] --> B{?}
B --> C{?}
B --> D[HashMap]
C --> E[ConcurrentHashMap]
C --> F[Collections.synchronizedMap]
D --> G{?}
G --> H[LinkedHashMap]
G --> I[TreeMap]
G --> D
```

12.3 Set选择 ★★★★★☆

12.4 Queue选择 ★★★★★☆

附录：面试高频问题速查

小林coding面试高频追问

Q1：HashMap既然线程不安全，为什么还要用？

参考答案：

- 单线程环境下，HashMap性能更好（无锁开销）
- 多线程环境下可用ConcurrentHashMap
- 低并发场景Collections.synchronizedMap足够
- 根据场景选择合适的集合

Q2：ConcurrentHashMap的分段锁已经被弃用了吗？

参考答案：

- JDK7使用Segment分段锁（ReentrantLock）
- JDK8改为CAS+synchronized，锁粒度更细
- "分段锁"概念在JDK8仍有，但实现方式变了
- JDK8通过synchronized锁头节点实现桶级锁

Q3：为什么ConcurrentHashMap不允许null值？

参考答案：

- null值会造成二义性：映射不存在 vs 映射值为null
- 多线程环境下无法区分
- HashMap允许null是历史原因，单线程场景可以区分

Q4：HashMap的负载因子为什么是0.75？

参考答案：

- 空间和时间复杂度的平衡
- 太小：频繁扩容，空间浪费
- 太大：碰撞增多，链表/红黑树变长
- 0.75是统计最优值，空间和时间效率最佳

Q5：红黑树和二叉平衡树的区别？

参考答案：

- 平衡二叉树（AVL）：严格平衡，插入删除开销大
- 红黑树：弱平衡，调整代价小
- HashMap场景：增删改查都频繁，红黑树综合性能更好

Q6：ArrayList和LinkedList选择哪个？

参考答案：

- ArrayList：频繁随机访问，尾部操作多
- LinkedList：频繁中间插入删除，需要队列/栈结构
- 现代JVM对数组连续内存优化好，ArrayList实际更快

Q7：CopyOnWriteArrayList的缺点？

参考答案：

- 每次写操作都复制数组，开销大
- 不适合写多读少场景
- 只能保证最终一致性，不保证实时一致
- 内存占用高（同时存在新旧数组）

>

笔记说明：本笔记根据黑马程序员Java面试课程和小林coding面试题整理，深入源码分析集合框架。使用mermaid图解便于理解，建议面试前快速回顾。

Java并发编程核心笔记

Java并发编程核心面试笔记

> 黑马程序员Java面试专题 + 小林coding面试题补充 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [线程基础](#1-线程基础)
2. [synchronized与Lock](#2-synchronized与lock)
3. [JMM内存模型](#3-jmm内存模型)
4. [volatile与CAS](#4-volatile与cas)
5. [AQS抽象队列同步器](#5-aqs抽象队列同步器)
6. [ReentrantLock可重入锁](#6-reentrantlock可重入锁)
7. [JUC并发工具类](#7-juc并发工具类)
8. [线程池原理](#8-线程池原理)
9. [并发容器](#9-并发容器)
10. [生产消费与线程安全](#10-生产消费与线程安全)
11. [死锁与排查](#11-死锁与排查)
12. [高频面试题](#12-高频面试题)

1. 线程基础

1.1 进程与线程的区别 ★☆☆☆☆

>

面试话术：进程是资源分配的基本单位，线程是CPU调度的基本单位。线程共享进程的堆和方法区，但有独立的程序计数器、虚拟机栈和本地方法栈。

1.2 并行与并发的区别 ★★☆☆☆

```
graph LR
    subgraph Concurrent
        A1[1] --> A2[A]
        A1 --> A3[B]
        A2 ----> A4[]
    end

    subgraph Parallel
        B1[1] --> B2[A]
        B3[2] --> B4[B]
        B2 --> B5[]
        B4 --> B5
    end
```

- **并发**：同一时间段内多个任务交替执行（CPU切换）
- **并行**：同一时刻多个任务真正同时执行（多核）

1.3 线程的创建方式 ★★★★★（高频）

```
// 1Thread
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread");
    }
}

// 2Runnable
class MyRunnable implements Runnable {
    @Override
```

```

    public void run() {
        System.out.println("Runnable");
    }
}

// 3Callable
class MyCallable implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        return 123;
    }
}

// 4
ExecutorService pool = Executors.newFixedThreadPool(10);
pool.execute(new RunnableTask());
Future<Integer> future = pool.submit(new MyCallable());

```

1.4 线程状态转换 ★★★★★ (高频)

```

stateDiagram-v2
    [*] --> : new Thread()
    --> : start()
    --> : CPU
    --> : yield()\nCPU
    --> : synchronized\nwait()/join()/sleep()
    --> : run()
    --> : \nnotify()/notifyAll()\n
    --> : notify()/notifyAll()\n
    --> [*]: interrupt()
    --> [*]: interrupt()

```

1.5 wait() vs sleep() vs yield() ★★★☆☆

> 面试话术：

- > - sleep()是Thread类的静态方法，让当前线程睡眠指定时间，不释放锁
- > - wait()是Object类的方法，会释放锁，需要通过notify/notifyAll唤醒
- > - yield()是Thread类的静态方法，让出CPU时间片，回到就绪状态

1.6 notify() vs notifyAll() ★★★☆☆

```

sequenceDiagram
    participant T1 as T1
    participant M as Monitor\n(Object)
    participant T2 as T2

    T1->>M: wait()
    Note over T1: \n
    T2->>M:
    T2->>M: notify()
    Note over M: \n->
    M->>T1:
    T1->>M:
    Note over T1: \n

```

- `notify()`：随机唤醒一个等待线程
- `notifyAll()`：唤醒所有等待线程，竞争锁

> 面试官：notify选择哪个线程？

>

>

候选人：notify在源码的注释中说到notify选择唤醒的线程是任意的，但是依赖于具体实现的JVM。JVM有很多实现，比较流行的就是HotSpot，HotSpot对notify()的实现并不是随机唤醒，而是"先进先出"的顺序唤醒。

1.7 线程间通信方式 ★★★★★☆

2. synchronized与Lock

2.1 synchronized底层原理 ★★★★★ (高频)

```

classDiagram
    class ObjectMonitor {
        +_owner: Thread
        +_WaitSet: Thread[]
        +_count: int
        +enter()
        +exit()
        +wait()
        +notify()
    }

    class MarkWord {
        +thread_id
        +age
        +biased_lock
        +lock
        +hashCode
    }

```

对象头结构：

monitorenter流程：

```

flowchart TD
    A[synchronized] --> B{mark word\n?}
    B -->|C| C{ID\nmark word?}
    C -->|D| D[]
    C -->|E| E{?}
    E -->|F| F[\n]
    E -->|G| G[]
    B -->|H| H{mCAS\n?}
    H -->|I| I[]
    H -->|J| J[\n]
    J --> K[\n]

```

2.2 锁的四种状态 ★★★★★（高频）

>

面试话术：JVM为了减少锁竞争带来的性能开销，设计了锁升级机制。偏向锁在无竞争情况下消除同步开销；轻量级锁采用CAS自旋避免线程阻塞；重量级锁让抢锁失败的线程阻塞。

2.3 synchronized锁升级过程 ★★★★★（高频）

```

flowchart TD
    A[synchronized] --> B{?}
    B -->|C| C{ID=MarkWordID?}
    C -->|D| D[\n]
    C -->|E| E{?}
    E -->|F| F[\n]
    E -->|G| G[]
    B -->|H| H{CAS\n?}
    H -->|I| I[]
    H -->|J| J{\n?}
    J -->|K| K[\n]
    J -->|G| G
    K --> L[\n]

```

具体过程：

1. ****无锁→偏向锁****：当一个线程首次进入synchronized块时，如果偏向锁启用，JVM会将Mark Word中的线程ID设为当前线程ID

2. ****偏向锁→轻量级锁****：当有其他线程尝试获取锁时，偏向锁升级。多个线程通过CAS竞争锁

3.

****轻量级锁→重量级锁****：自旋超过10次或自旋线程过多时，轻量级锁膨胀为重量级锁，未获取锁的线程被阻塞

2.4 JVM对synchronized的优化 ★★★★★☆

2.5 synchronized vs ReentrantLock ★★★★★（高频）

```

// synchronized
synchronized (obj) {
    //
}

```

```
// ReentrantLock
ReentrantLock lock = new ReentrantLock(true); // true
lock.lock();
try {
    //
} finally {
    lock.unlock();
}
}
```

3. JMM内存模型

3.1 JMM八种内存交互操作 ★★★☆☆

```
graph TB
    subgraph Main_Memory [Main Memory]
        M[M]
    end
    subgraph Thread_Working_Memory [Thread Working Memory]
        T1[T1]
        T2[T2]
    end

    T1 -->|read/load| M
    M -->|read/load| T1
    T1 -->|store/write| M
    M -->|store/write| T1

    T2 -->|read/load| M
    M -->|read/load| T2
    T2 -->|store/write| M
    M -->|store/write| T2
```

3.2 happens-before八大规则 ★★★★★☆

4. volatile与CAS

4.1 volatile特性 ★★★★★ (高频)

```
flowchart LR
    subgraph A
        A1[A1[volatile\nflag=true]]
        A2[A2[Store Barrier\n]]
    end
    subgraph M
        M[M[flag=true]]
    end
    subgraph B
        B1[B1[Load Barrier\n]]
        B2[B2[flag\nflag=true]]
    end

    A1 --> A2 --> M --> B1 --> B2
```

两大特性：

1. ****可见性****：写操作后立即刷新到主内存，读操作从主内存读取
2. ****有序性****：禁止指令重排序（内存屏障）

4.2 CAS原理 ★★★★★ (高频)

```
flowchart TD
    A["A[=A]"] --> B["B{compare\nA==?}"]
    B -->|true| C["C[Swap\nB]"]
    B -->|false| D["D[\n]"]
    C --> E["E[\ntrue]"]
    D --> F["F[\nfalse]"]
```


CAS三大问题：

5. AQS抽象队列同步器

5.1 AQS核心结构 ★★★★★☆

```
classDiagram
    class AbstractQueuedSynchronizer {
        +state: int
        +head: Node
        +tail: Node
        +acquire()
        +release()
        +tryAcquire()
        +tryRelease()
    }

    class Node {
        +waitStatus: int
        +prev: Node
        +next: Node
        +thread: Thread
        +SHARED: Node
        +EXCLUSIVE: Node
    }

    AbstractQueuedSynchronizer --> Node :
```

Node状态：

5.2 AQS核心思想 ★★★★★★（高频）

> 面试官：介绍一下AQS

>

>

候选人：AQS全称为AbstractQueuedSynchronizer，是Java中的一个抽象类，是用于构建锁、同步器、协作工具类的工具类（框架）。

>

>

AQS核心思想是，如果被请求的共享资源空闲，那么就将当前请求资源的线程设置为有效的工作线程，将共享资源设置为锁定状态；如果共享资源被占用，就需要一定的阻塞等待唤醒机制来保证锁分配。这个机制主要用的是CLH队列的变体实现的，将暂时获取不到锁的线程加入到队列中。

>

>

CLH：Craig、Landin and Hagersten队列，是单向链表，AQS中的队列是CLH变体的虚拟双向队列（FIFO），AQS是通过将每条请求共享资源的线程封装成一个节点来实现锁的分配。

>

>

AQS使用一个Volatile的int类型的成员变量来表示同步状态，通过内置的FIFO队列来完成资源获取的排队工作，通过CAS完成对State值的修改。

5.3 AQS三大部分 ★★★★★★（高频）

5.4 独占模式获取流程 ★★★★★☆

```
flowchart TD
    A[acquire] --> B{tryAcquire\n?}
    B -->|C[\n]| C
    B -->|D[addWaiter\nNode]| D
    D --> E{?}
    E -->|F{tryAcquire\n?}| F
    E -->|G[shouldParkAfterFailedAcquire\n]| G
    F -->|H[setHead\n]| H
    G -->|I[setHead\n]| I
    H --> I
    I --> H
```

H --> I[\n]

5.5 CAS和AQS的关系 ★★★★★☆

> 面试官：CAS 和 AQS 有什么关系？

>

> 候选人：CAS 和 AQS 两者的区别：

>

CAS

是一种乐观锁机制，它包含三个操作数：内存位置（V）、预期值（A）和新值（B）。整个过程是原子性的。

> - AQS 是一个用于构建锁和同步器的框架，许多同步器如 ReentrantLock、Semaphore、CountDownLatch 等都是基于 AQS 构建的。

>

> 两者的联系：

> - CAS 为 AQS 提供原子操作支持：AQS 内部使用 CAS 操作来更新 state 变量，以实现线程安全的状态修改。

6. ReentrantLock可重入锁

6.1 可重入原理 ★★★★★☆

```
flowchart LR
    subgraph T
        A1[tryAcquire\nstate=1] --> A2[\nstate+1]
        A2 --> A3[\nstate+1]
        A3 --> A4[release\nstate-1]
        A4 --> A5[release\nstate=0\n]
    end
    end
```

> 原理：ReentrantLock内部维护一个state计数器，重入一次+1，释放一次-1，直到state=0才完全释放。

6.2 公平锁 vs 非公平锁

```
//
ReentrantLock lock1 = new ReentrantLock();

//
ReentrantLock lock2 = new ReentrantLock(true);
```

> 面试话术：非公平锁性能更高，因为可以减少线程切换开销。但可能造成线程饥饿。synchronized是非公平锁。

6.3 ReentrantLock高级特性 ★★★★★☆

7. JUC并发工具类

7.1 CountDownLatch ★★★★★★（高频）

> 面试官：CountDownLatch 是做什么的讲一讲？

>

> 候选人：CountDownLatch 是 Java 并发包（java.util.concurrent）中的一个同步工具类，用于让一个或多个线程等待其他线程完成操作后再继续执行。

>

> 核心原理：

> - 初始化计数器：创建 CountDownLatch 时指定一个初始计数值（如 N）

> - 等待线程阻塞：调用 await() 的线程会被阻塞，直到计数器变为 0

> - 任务完成通知：其他线程完成任务后调用 countDown()，使计数器减 1

> - 唤醒等待线程：当计数器减到 0 时，所有等待的线程会被唤醒

```
//
public class CountDownLatchDemo {
    public static void main(String[] args) throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(3);

        for (int i = 0; i < 3; i++) {
```

```

        new Thread(() -> {
            System.out.println("" + Thread.currentThread().getName() + "");
            latch.countDown();
        }).start();
    }

    latch.await(); //
    System.out.println("");
}
}

```

vs CyclicBarrier :

7.2 CyclicBarrier ★★★★★☆

```

//
public class CyclicBarrierDemo {
    public static void main(String[] args) {
        CyclicBarrier barrier = new CyclicBarrier(3, () -> {
            System.out.println("");
        });

        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                System.out.println("" + Thread.currentThread().getName() + "");
                try {
                    barrier.await(); //
                } catch (InterruptedException | BrokenBarrierException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}

```

7.3 Semaphore ★★★★★☆ (高频)

> 面试官：Semaphore 是什么？用在什么场景？

>

> 候选人：Semaphore 是一个计数信号量，用于控制同时访问某个共享资源的线程数量。通过 `acquire()` 方法获取许可，使用 `release()` 方法释放许可。如果没有许可可用，线程将被阻塞，直到有许可被释放。可以用来限制对某些资源（如数据库连接池、文件操作等）的并发访问量。

```

// 3
public class SemaphoreDemo {
    public static void main(String[] args) {
        Semaphore semaphore = new Semaphore(3);

        for (int i = 0; i < 10; i++) {
            new Thread(() -> {
                try {
                    semaphore.acquire(); //
                    System.out.println("" + Thread.currentThread().getName() + "");
                    Thread.sleep(1000);
                    System.out.println("" + Thread.currentThread().getName() + "");
                    semaphore.release(); //
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }).start();
        }
    }
}

```

7.4 JUC并发工具类对比 ★★★★★☆

8. 线程池原理

8.1 线程池七大参数 ★★★★★☆ (高频)

```

flowchart TD
    A[] --> B{corePoolSize\n}
    B -->| | C[\n]
    B -->| | D{\nworkQueue}
    D -->| | E[\n]
    D -->| | F{maximumPoolSize\n}
    F -->| | G[\n]
    F -->| | H[]

    C --> I[\n]
    E --> C
    G --> I

```

8.2 四种拒绝策略 ★★★★★（高频）

8.3 Executors创建线程池的坑 ★★★★★☆

```

// 100M
ExecutorService fixedPool = Executors.newFixedThreadPool(100);
// Integer.MAX_VALUEOOM

// 2
ExecutorService cachedPool = Executors.newCachedThreadPool();
// maximumPoolSize=Integer.MAX_VALUE

//
new ThreadPoolExecutor(
    corePoolSize,
    maximumPoolSize,
    keepAliveTime,
    TimeUnit.SECONDS,
    new LinkedBlockingQueue<>(100), //
    new ThreadFactoryBuilder().setNameFormat("pool-%d").build(),
    new ThreadPoolExecutor.AbortPolicy()
);

```

8.4 常见线程池类型 ★★★★★☆☆

9. 并发容器

9.1 ConcurrentHashMap vs Hashtable vs HashMap

9.2 CopyOnWriteArrayList原理 ★★★★★☆☆

```

flowchart TD
    A[] --> B[\n]
    C[] --> D[]
    D --> E[]
    E --> F[]
    F --> G[GC]

    style B fill:#90EE90
    style C fill:#FFB6C1
    style D fill:#FFB6C1

```

特点：

- 读操作不加锁，写操作复制数组
- 适用于读多写少场景
- 写操作有内存开销

10. 生产消费与线程安全

10.1 生产者消费者模式 ★★★★★☆☆

```

flowchart LR
    subgraph
        P1[]
    end

```

```

subgraph
    B[BlockingQueue]
end

subgraph
    C1[]
end

P1 -->|offer| B
B -->|poll| C1

```

实现方式一：BlockingQueue

```

public class ProducerConsumerDemo {
    private BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(10);

    //
    public void produce() {
        while (true) {
            int data = new Random().nextInt(100);
            try {
                queue.offer(data, 2, TimeUnit.SECONDS);
                System.out.println("" + data);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    //
    public void consume() {
        while (true) {
            try {
                Integer data = queue.poll(2, TimeUnit.SECONDS);
                System.out.println("" + data);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

```

实现方式二：synchronized + wait/notify

```

public class ProducerConsumerSync {
    private int buffer = 0;
    private boolean hasData = false;

    public synchronized void produce(int data) {
        while (hasData) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        buffer = data;
        hasData = true;
        System.out.println("" + data);
        notifyAll();
    }

    public synchronized int consume() {
        while (!hasData) {
            try { wait(); } catch (InterruptedException e) { e.printStackTrace(); }
        }
        hasData = false;
        notifyAll();
        return buffer;
    }
}

```

实现方式三：Lock + Condition

```

public class ProducerConsumerLock {
    private int buffer = 0;
    private boolean hasData = false;
    private Lock lock = new ReentrantLock();
    private Condition notEmpty = lock.newCondition();
    private Condition notFull = lock.newCondition();
}

```

```

public void produce(int data) {
    lock.lock();
    try {
        while (hasData) {
            notFull.await();
        }
        buffer = data;
        hasData = true;
        System.out.println(""" + data);
        notEmpty.signalAll();
    } finally {
        lock.unlock();
    }
}

public int consume() {
    lock.lock();
    try {
        while (!hasData) {
            notEmpty.await();
        }
        hasData = false;
        notFull.signalAll();
        return buffer;
    } finally {
        lock.unlock();
    }
}
}

```

10.2 CountdownLatch vs CyclicBarrier

11. 死锁与排查

11.1 死锁的必要条件 ★★★★★ (高频)

> 面试官：什么是死锁？死锁的必要条件是什么？

>

>

候选人：死锁是指两个或两个以上的线程在执行过程中，因争夺资源而造成的互相等待现象，如果没有外力干预，它们都无法推进下去。

>

> 死锁的四个必要条件：

- > 1. 互斥条件：一个资源每次只能被一个线程使用
- > 2. 请求与保持条件：一个线程因请求资源而阻塞时，对已获得的资源保持不放
- > 3. 不剥夺条件：线程已获得的资源，在未使用完之前，不能被强行剥夺
- > 4. 循环等待条件：若干线程形成头尾相接的循环等待资源关系

11.2 手写死锁代码 ★★★★★☆

```

public class DeadLockDemo {
    private static final Object lockA = new Object();
    private static final Object lockB = new Object();

    public static void main(String[] args) {
        // 1lockAlockB
        Thread t1 = new Thread(() -> {
            synchronized (lockA) {
                System.out.println("1lockA");
                try { Thread.sleep(100); } catch (InterruptedException e) {}
                synchronized (lockB) {
                    System.out.println("1lockB");
                }
            }
        });

        // 2lockBlockA
        Thread t2 = new Thread(() -> {

```

```

        synchronized (lockB) {
            System.out.println("2lockB");
            try { Thread.sleep(100); } catch (InterruptedException e) {}
            synchronized (lockA) {
                System.out.println("2lockA");
            }
        }
    });

    t1.start();
    t2.start();
}
}

```

11.3 避免死锁的方法 ★★★★★☆

```

//
public class DeadLockSolution {
    public static void main(String[] args) {
        // 12 lockA -> lockB
    }
}

```

11.4 死锁排查命令 ★★★★★☆

```

# 1. Java
jps -l

# 2.
jstack -l <pid>

# 3.
jstack -l <pid> | grep -A 10 "deadlock"

```

12. 高频面试题

12.1 synchronized相关 ★★★★★★

> 面试官：synchronized 锁升级的过程讲一下

>

> 候选人：具体的锁升级的过程是：无锁→偏向锁→轻量级锁→重量级锁。

>

> 1. 无锁：没有开启偏向锁时的状态

>

偏向锁：如果还没有一个线程拿到这个锁，状态为"匿名偏向"。当一个线程拿到偏向锁时，下次竞争只需比较线程ID

> 3. 轻量级锁：多个线程通过CAS竞争锁，通过自旋避免线程阻塞

> 4. 重量级锁：自旋超过阈值后升级为重量级锁，未获取锁的线程被操作系统挂起

> 面试官：synchronized 和 ReentrantLock 区别？

>

> 候选人：

> - 用法不同：synchronized 可修饰方法/代码块，ReentrantLock 只能用在代码块

> - 获取释放锁：synchronized自动释放，ReentrantLock需手动

> - 锁类型：synchronized非公平锁，ReentrantLock可配置公平/非公平

> - 响应中断：ReentrantLock可响应中断解决死锁，synchronized不能

> - 底层实现：synchronized是JVM Monitor实现，ReentrantLock是AQS实现

> 面试官：synchronized 支持重入吗？如何实现的？

>

> 候选人：synchronized是可重入锁。底层利用线程ID和锁状态status实现：

- > - 线程第一次获取锁时，状态从0变为1，线程ID存储自己的ID
- > - 同一线程再次获取时，比较线程ID是自己的，则status+1
- > - 释放时status-1，直到为0时完全释放锁

12.2 线程池相关 ★★★★★

- > 面试官：线程池参数有哪些？
- >
- > 候选人：线程池有7个核心参数：
- > 1. corePoolSize：核心线程数
- > 2. maximumPoolSize：最大线程数
- > 3. keepAliveTime：空闲线程存活时间
- > 4. unit：时间单位
- > 5. workQueue：任务队列
- > 6. threadFactory：线程工厂
- > 7. handler：拒绝策略
- >
- > 执行流程：任务提交→核心线程→队列→最大线程→拒绝策略

- > 面试官：线程池的拒绝策略有哪些？
- >
- > 候选人：有四种：
- > 1. AbortPolicy：抛RejectedExecutionException，默认策略
- > 2. CallerRunsPolicy：由调用线程执行
- > 3. DiscardPolicy：直接丢弃
- > 4. DiscardOldestPolicy：丢弃最老的任务

12.3 JUC并发工具类 ★★★★★

- > 面试官：CountDownLatch、CyclicBarrier、Semaphore的区别？
- >
- > 候选人：
- > - CountDownLatch：倒数计数，计数器为0时放行，不可复用
- > - CyclicBarrier：栅栏，所有线程都到达后一起放行，可复用
- > - Semaphore：信号量，控制同时访问资源的线程数量

12.4 ThreadLocal相关 ★★★★★☆

- > 面试官：ThreadLocal会有什么问题？如何解决？
- >
- > 候选人：ThreadLocal可能导致内存泄漏：
- > - ThreadLocalMap使用ThreadLocal的弱引用作为key
- > - 如果ThreadLocal没有外部强引用，GC时会回收，但value仍被Entry持有
- > - 解决方法是使用完ThreadLocal后调用remove()方法

12.5 线程通信相关 ★★★★★☆

- > 面试官：线程间通信方式有哪些？
- >
- > 候选人：
- > 1. Object的wait/notify/notifyAll
- > 2. Lock和Condition
- > 3. volatile关键字
- > 4. CountDownLatch/CyclicBarrier/Semaphore
- > 5. BlockingQueue
- > 6. Thread.join()

附录：面试高频问题速查

>

笔记说明：本笔记根据黑马程序员Java面试课程和小林coding面试题整理，涵盖并发编程核心面试知识点。配合mermaid图解便于理解，建议面试前快速回顾。

操作系统核心面试笔记

操作系统核心面试知识点

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [进程与线程区别](#1-进程与线程区别)
2. [进程状态转换](#2-进程状态转换)
3. [进程调度算法](#3-进程调度算法)
4. [死锁原因与处理](#4-死锁原因与处理)
5. [内存管理](#5-内存管理)
6. [页面置换算法](#6-页面置换算法)
7. [高频面试题](#7-高频面试题)

1. 进程与线程区别 ★★☆☆

核心对比

为什么线程切换比进程快？

- 线程共享进程的代码段、数据段、打开的文件等资源
- 线程切换只需切换寄存器和栈，不涉及地址空间切换
- 进程切换需要切换页表、切换内核态堆栈

进程间通信方式

2. 进程状态转换 ★★★

五状态模型

stateDiagram-v2

```
[*] --> :  
--> : CPU  
--> :  
--> :  
--> : I/O  
--> : I/O  
--> :  
--> [*]:
```

状态说明

就绪态 vs 阻塞态

- ****就绪态****：只缺少CPU资源，其他条件已满足
- ****阻塞态****：缺少的不止是CPU，可能在等I/O、信号量等

3. 进程调度算法 ★★★★★

批处理系统算法

交互系统算法

多级反馈队列调度原理

```
flowchart TD
    A[] --> B{?}
    B --> C[]
    B --> D[]
    D --> E{?}
    E --> F[]
    E --> G[]
    F --> H[...]
```

时间片大小选择

- ****时间片太大****：响应时间差，退化为FCFS
- ****时间片太小****：上下文切换频繁，系统开销大
- ****一般选择****：让80%的进程在时间片内完成

4. 死锁原因与处理 ★★★★★

死锁产生的必要条件

死锁处理策略

```
flowchart LR
    A[] --> B[]
    A --> C[]
    A --> D[]
    A --> E[]

    B --> B1[]
    B --> B2[]
    B --> B3[]
    B --> B4[]

    C --> C1[]
    C --> C2[]

    D --> D1[]
    D --> D2[/]/

    E --> E1[Linux]
```

银行家算法

核心思想：在资源分配前检查分配后是否仍处于安全状态

安全状态判断：存在一个安全序列使得所有进程都能完成

死锁与饥饿的区别

哲学家就餐问题解决方案

1. 最多允许4个哲学家同时拿左筷子
2. 奇数号先拿左再拿右，偶数号先拿右再拿左
3. 检测到邻居在用餐时不拿筷子

5. 内存管理 ★★★★★

分页管理

= +
= +

```
flowchart LR
    A[] --> B[]
    A --> C[]
    B --> D[]
    D --> E[]
    E --> F[]
```

```
C --> G[]
F --> G
```

分段管理

= +

虚拟内存

核心思想：程序不必全部加载到内存，运行时只加载需要的部分

TLB作用

```
flowchart TD
    A[CPU] --> B[MMUTLB]
    B --> C{TLB?}
    C --> D[]
    C --> E[]
    E --> F[]
    F --> G[TLB]
    D --> H[]
    G --> H
```

TLB命中率：可达90%以上，大大减少内存访问时间

6. 页面置换算法 ★★★★★

算法分类

FIFO Belady异常

```
flowchart LR
    A[=3] --> B[=9]
    A --> C[=4]
    C --> D[=10]
    D --> E[Belady]
```

Belady异常：增加物理页框数反而导致缺页率增加

Clock算法原理

1. 循环扫描页面
2. 访问位=0时置换
3. 访问位=1时清零并继续
4. 改进版增加修改位考虑

页面抖动

- **定义**：进程频繁调入调出页面
- **原因**：进程数过多/物理内存不足
- **解决**：调节进程数或增加内存

7. 高频面试题 ★★★★★~★★★★★

Q1：进程和线程的区别？

参考答案：

- 进程是资源分配的基本单位，线程是CPU调度的基本单位
- 进程有独立地址空间，线程共享进程地址空间
- 进程切换开销大（切换页表、内核栈），线程切换开销小
- 线程间通信更高效（共享内存），进程间通信需要IPC

Q2：什么是死锁？如何避免？

参考答案：

死锁是多个进程因循环等待资源而无法继续执行的现象。

必要条件：互斥条件、占有并等待、不可抢占、循环等待

处理方法：

1. **预防**：破坏其中一个条件（如资源排序）
2. **避免**：银行家算法检查安全状态
3. **检测恢复**：定期检测并剥夺/回滚
4. **忽略**：如Linux默认策略

Q3：进程调度算法有哪些？哪个最好？

参考答案：

- 批处理：FCFS、SJF、SRTF
- 交互系统：RR、优先级、多级反馈队列
- 实时系统：EDF、RMS

没有"最好"：不同场景适合不同算法

- RR适合通用系统（公平）
- 多级反馈队列是综合最优（Unix使用）
- 实时系统用EDF/RMS

Q4：解释一下虚拟内存？

参考答案：

- 虚拟内存让程序访问大于物理内存的地址空间
- 通过MMU将虚拟地址映射到物理地址
- 不常用的页面存储在磁盘（swap）
- 访问不在内存的页面时触发缺页中断，从磁盘调入
- 实现了内存扩展和进程地址空间隔离

Q5：页面置换算法详解

参考答案：

- **OPT**：置换最长时间不再访问的页（理论最优）
- **FIFO**：先进先出，简单但可能Belady异常
- **LRU**：置换最近最久未使用的页，性能好但需硬件支持
- **Clock**：二次chance算法，用访问位近似LRU

Q6：并发和并行的区别

参考答案：

- **并发(Concurrency)**：多个任务交替执行，共享CPU时间片
- **并行(Parallelism)**：多个任务同时执行，需要多CPU核心

> 并发是交替执行，并行是同时执行

Q7：什么是线程安全？如何实现？

参考答案：

线程安全指多个线程访问共享资源时不会产生不确定结果。

实现方法：

1. **互斥锁**：mutex、semaphore
2. **无锁编程**：原子操作（CAS）
3. **避免共享**：线程本地存储（TLS）
4. **不可变对象**：对象状态不可修改

Q8：什么是上下文切换？

参考答案：

CPU从一个进程/线程切换到另一个时，保存当前进程状态并恢复另一个进程状态的过程。

保存内容包括：寄存器值、程序计数器、栈指针、进程控制块(PCB)

Q9：什么是快表(TLB)？作用是什么？

参考答案：

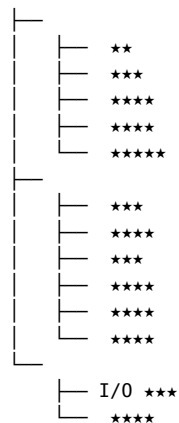
TLB(Translation Lookaside Buffer)是MMU中的硬件缓存。

作用：

- 缓存最近使用的页表项
- 避免每次访存都查页表（减少一次内存访问）
- 命中率可达90%以上
- 加速虚拟地址到物理地址的转换

Q10：系统调用和库函数的区别？

附录：知识图谱



> 笔记说明：本笔记结合小林coding和黑马程序员整理，涵盖操作系统核心面试知识点。建议面试前快速回顾。

消息队列核心面试笔记

消息队列核心面试知识点

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [消息队列应用场景](#1-消息队列应用场景)
2. [Kafka与RabbitMQ对比](#2-kafka与rabbitmq对比)
3. [Kafka高吞吐量原因](#3-kafka高吞吐量原因)
4. [消息丢失、重复消费、顺序性](#4-消息丢失重复消费顺序性)
5. [消费者组与分区](#5-消费者组与分区)
6. [高频面试题](#6-高频面试题)

1. 消息队列应用场景 ★★★

为什么需要消息队列？

消息队列（MQ）核心解决系统架构三大痛点：解耦、异步、削峰

flowchart TB

A[] --> B[]

A --> C[]

A --> D[]

B --> B1[\n]

C --> C1[\n]

D --> D1[MQ\n]

消息队列的缺点 ★★★

> 没有银弹，引入MQ是典型的Trade-off

1. **可用性降低**：中间件宕机导致整个链路瘫痪，必须搭建高可用集群
2. **开发复杂性上升**：需处理消息重复、消息丢失、分布式事务等问题
3. **数据一致性**：A系统成功、B系统失败的场景需要分布式事务方案

四大主流MQ对比选型 ★★★

选型建议：

- 高吞吐场景（日志、大数据）→ **Kafka**
- 电商等复杂业务，需要事务消息、延迟队列 → **RocketMQ**
- 多主题灵活路由 → **RabbitMQ**

2. Kafka与RabbitMQ对比 ★★★

架构模型对比

文件存储模型区别（高频考点）★★★★

Kafka：独立分区文件设计

- 每个分区独立小本子，顺序写高效
- 问题：Topic过多时，退化为随机写，性能暴跌

RocketMQ：CommitLog混合存储（阿里自研解决海量Topic问题）

- 所有消息顺序追加写入统一大文件
- 后台线程整理出ConsumeQueue索引目录

- 保证海量Topic下依然是顺序写

消息确认机制对比 ★★★★★

Kafka：偏移量(Offset)提交模式

```
"100"
```

```
991000offset99
```

RocketMQ：单条消息状态确认

```
//
CONSUME_SUCCESS //
RECONSUME_LATER //
```

RocketMQ重试机制：阶梯式延迟（10s→30s→1min），十几次失败后进入死信队列(DLQ)

```
---
```

3. Kafka高吞吐量原因 ★★★★★

核心技术点

零拷贝技术原理

```
flowchart LR
    subgraph
        A1[] --> A2[]
        A2 --> A3[]
        A3 --> A4[Socket]
        A4 --> A5[]
    end

    subgraph
        B1[] --> B2[]
        B2 --> B3[Socket]
        B3 --> B4[]
    end

    A1 -. ->|2| B1
```

```
---
```

4. 消息丢失、重复消费、顺序性 ★★★★★

4.1 消息丢失问题（必考题） ★★★★★

三环节保证不丢失：

```
flowchart LR
    A[] --> B[MQ]
    B --> C[]

    A --> A1[\\n]
    B --> B1[\\nacks=all]
    C --> C1[ACK\\nACK]
```

Kafka的acks机制：

- `acks=0`：发出去不管，数据可能丢失
- `acks=1`：仅Leader写入，不保证从节点同步
- `acks=all/-1`：必须等所有ISR同步完成

4.2 重复消费与幂等性（高频追问） ★★★★★

产生原因：生产者重试、网络抖动导致重复发送

解决思路：消费者端幂等设计

```
---
```

```
CREATE TABLE consumed_messages (
    message_id VARCHAR(64) PRIMARY KEY,
    status TINYINT DEFAULT 0,
    created_at TIMESTAMP
);
```

```
-- status=0status=1
```


4.3 消息顺序性保证 ★★★★★

Kafka天然保证分区内有序，跨分区无序

保证顺序的方案：

1. ****生产者端****：相同Key的消息发送到同一分区
2. ****消费者端****：单线程消费同一分区

RocketMQ顺序消息原理：

- 生产者把有序消息发到同一MessageQueue
- Broker对MessageQueue加锁
- 消费者串行消费

5. 消费者组与分区 ★★★★★

核心规则

> 一个分区同一时刻只能被消费者组中的一个消费者消费

Topic: order-topic (4)

Consumer Group: order-group (3)

- A → 01
- B → 2
- C → 3

分区数与消费者数关系

最佳实践：消费者数 = 分区数（最大化吞吐）

消费者组优点

- ****水平扩展****：加机器提升消费能力
- ****故障容错****：宕机消费者自动重新分配分区

6. 高频面试题 ★★★★★~★★★★★

Q1：消息队列为什么选择RocketMQ？

参考答案：

1. ****Java语言开发****：易上手，源码可读性强
2. ****社区活跃****：阿里开源，双十一验证，特性丰富
3. ****功能强大****：支持事务消息、延迟队列、消息回溯等
4. ****高吞吐量****：单机10万级TPS

Q2：Kafka为什么这么快？

参考答案：

1. ****顺序写入****：消息追加到磁盘日志，减少寻道时间
2. ****批量处理****：积累一定量才发送，减少IO次数
3. ****零拷贝****：sendfile技术，内核直接传输
4. ****压缩传输****：减少网络数据量
5. ****分区并行****：水平扩展

Q3：如何处理消息积压？

参考答案：

小量积压：扩容消费者（前提：分区数足够）

海量积压（几百万消息几小时）：

1. bug
2. Topicpartition10
3. Topic → Topic

4. 10Topic
5.

业务允许时的急救措施：直接重置消费位点到最新，跳过积压消息

Q4：如何保证消息不重复消费？

参考答案：

```
// 1. Redis/  
String key = "consumed:" + messageId;  
if (redis.exists(key)) {  
    return; //  
}  
  
// 2. ...  
  
// 3.  
redis.setex(key, 86400, "1");
```

Q5：RocketMQ事务消息原理？

参考答案：

```
sequenceDiagram  
    participant P as  
    participant M as MQ  
    participant C as  
  
    P->>M: 1.Half Message  
    M-->>P: 2.ACK  
    P->>P: 3.  
    P->>M: 4.commit/rollback  
    M->>M: 5.  
    M->>C: 6.
```

核心思想：最终一致性，不是强一致性。RocketMQ保证的是"本地事务执行成功 → 消息一定能被消费"

Q6：RabbitMQ核心组件有哪些？

参考答案：

- **Producer**：生产者，发送消息到交换机
- **Consumer**：消费者，从队列取消息
- **Queue**：队列，真正存储消息（FIFO）
- **Exchange**：交换机，根据路由规则分发消息
- **RoutingKey + Binding**：交换机与队列的绑定纽带

四种交换机类型：

Q7：让你设计一个消息队列，如何架构？

参考答案：

考察点：架构设计能力、高可用、可扩展性、幂等性

```
1. Producer → Broker → Consumer → ACK  
2. RPCDubbo  
3. Kafka+  
4. zk  
5. + ACK +  
6. ID  
7. → Leader/Follower →  
8. +  
9. Broker → Topic → PartitionPartition
```

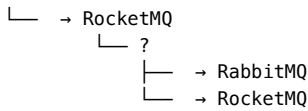
Q8：Kafka和RocketMQ如何选型？

参考答案：

- **Kafka**：高吞吐、低延迟，适合大数据日志采集（日志、埋点）
- **RocketMQ**：功能丰富，支持事务消息、延迟队列，Java技术栈首选

选型决策树：

```
/?  
├─ → Kafka
```



Q9：RabbitMQ死信队列和延迟队列是什么？

参考答案：

延迟队列：消息不立即被消费，等指定时间后才投递

- 场景：订单30分钟未支付自动取消
- 实现：x-delayed-message类型交换机

死信队列：无法正常处理的消息的兜底方案

- 三种情况变死信：消费者拒绝且不回队列、消息过期、队列满了
- 死信交换机自动转发到DLQ
- 后续：记录日志、人工处理或重试

Q10：如何保证Kafka消息不丢失？

参考答案：

1. **生产者端**：使用`acks=all`，等待所有ISR副本确认
2. **Broker端**：设置`replication.factor >= 3`，`min.insync.replicas >= 2`
3. **消费者端**：关闭自动提交Offset，改为手动提交，确认处理完成后提交

附录：面试核心知识点星级

> 笔记说明：本笔记结合小林coding和黑马程序员整理，涵盖消息队列核心面试知识点。建议面试前快速回顾。

分布式核心面试笔记

分布式核心面试知识点

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [CAP定理](#1-cap定理)
2. [BASE理论](#2-base理论)
3. [一致性协议](#3-一致性协议)
4. [负载均衡策略](#4-负载均衡策略)
5. [分布式锁与分布式ID](#5-分布式锁与分布式id)
6. [分布式事务](#6-分布式事务)
7. [分布式限流算法](#7-分布式限流算法)
8. [高频面试题](#8-高频面试题)

1. CAP定理 ★★★

CAP原则

```
flowchart TB
    A[CAP] --> B[Consistency]
    A --> C[Availability]
    A --> D[Partition tolerance]

    B --> B1[]
    C --> C1[]
    D --> D1[]
```

CAP原则：在一个分布式系统中，一致性、可用性、分区容错性三者不可得兼。

CAP三选二

为什么必须选P？

- 网络分区在分布式系统中不可避免
- 分区时必须在C和A之间做出选择

2. BASE理论 ★★★

BASE理论核心思想

核心思想：既然强一致性太难做到，那就牺牲短暂的一致性，换取系统的高可用和高性能。

```
flowchart TB
    A[BASE] --> B[]
    A --> C[]
    A --> D[]

    B --> B1["B1[\\n]"]
    C --> C1["C1[\\n]"]
    D --> D1["D1[\\n]"]
```

三个概念

BASE vs CAP

- **CAP**：强一致性，任何时刻都要一致
- **BASE**：允许暂时不一致，最终一致即可

- ****实践****：绝大多数互联网架构采用BASE，如消息队列异步处理、缓存等

3. 一致性协议 ★★★★★

3.1 Raft算法

核心思想：把一致性过程拆成三个部分：选主、日志复制、安全性保证

```
flowchart TB
    subgraph Raft
        A[Leader] --> B[]
        A --> C[Follower]
        C --> D[]
        D --> E[]
    end

    subgraph
        F[] --> G[]
        G --> H[]
        H --> F
    end
```

选主过程：

1. 跟随者一段时间没收到领导者的心跳，变成候选人
2. 候选人给自己投票，再向其他节点拉票
3. 拿到****超过半数的投票****成为新的领导者

日志复制：

1. 客户端请求发给领导者，领导者把操作写成日志
2. 同步给所有跟随者，等****超过半数节点确认****
3. 领导者提交，通知所有跟随者一起提交

应用场景：etcd、Kubernetes等

3.2 Paxos协议

核心思想：解决"多个节点协同，最终达成同一个决策"的问题

三个核心角色：

- ****提议者 (Proposer) ****：发起提案的节点
- ****接受者 (Acceptor) ****：负责接收提案、投票决定
- ****学习者 (Learner) ****：不参与投票，只同步最终结果

两阶段提交流程：

```
sequenceDiagram
    participant P as
    participant A as 1
    participant B as 2
    participant L as

    Note over P:
    P->>A: N
    P->>B: N
    A-->>P: N
    B-->>P: N

    Note over P:
    P->>A: N, V
    P->>B: N, V
    A-->>P:
    B-->>P:
    P->>L:
```

3.3 ZAB协议

核心：ZooKeeper专门设计的分布式一致性协议

集群节点角色：

- ****Leader****：接收写请求、发指令

- ****Follower****：参与投票和数据同步
- ****Observer****：不参与投票，只同步数据

两种工作模式：

Raft vs Paxos vs ZAB对比

4. 负载均衡策略 ★★★

常见负载均衡算法

实际应用：Nginx、Dubbo、Spring Cloud Ribbon等

5. 分布式锁与分布式ID ★★★★★

5.1 分布式锁

Redis实现分布式锁：

```
flowchart LR
    A[] --> B["B{SET lock_key unique NX PX 10000}"]
    B --> C[]
    C --> D["D[Lua]"]
    D --> E[]
```

三个条件：

1. ****加锁原子性****：`SET lock\_key unique\_value NX PX 10000`
2. ****锁过期时间****：避免客户端异常无法释放锁
3. ****唯一标识****：值使用UUID，确保只有加锁客户端才能解锁

Zookeeper实现分布式锁：

1. 创建持久节点`/locks`
2. 创建临时有序节点
3. 判断当前节点是否为最小节点：如果是，获取锁；如果不是，监听前一个节点
4. 业务完成后delete当前节点释放锁

对比：Zookeeper是强一致性（CP），Redis是最终一致性性能更好

5.2 分布式ID

什么时候需要分布式ID？

分布式ID生成方案：

雪花算法原理：

$64 = (41) + (10) + (12)$

- 开头存时间戳，ID趋势递增
- 中间存机器号，全网不重复
- 末尾存序列号，支持同一毫秒内多ID

6. 分布式事务 ★★★★★

什么情况下需要分布式事务？

- ****微服务跨服务协同****：订单服务+库存服务+账户服务，必须同时成功或回滚
- ****分库分表****：一个方法内操作多个数据库实例

分布式事务解决方案对比

2PC两阶段提交

```
sequenceDiagram
    participant C as
```

```
participant P1 as 1
participant P2 as 2
```

```
Note over C,P2:
C->>P1:
C->>P2:
P1-->>C:
P2-->>C:
```

```
Note over C,P2:
C->>P1:
C->>P2:
P1-->>C:
P2-->>C:
```

缺点：单点故障、资源锁导致并发性能低

TCC模式

```
flowchart TB
    A[TCC] --> B[Try]
    B --> C[Confirm]
    B --> D[Cancel]

    C --> E[]
    D --> F[]
```

适用场景：电商订单、库存管理等高并发场景

本地消息表 + MQ

```
flowchart LR
    A[] --> B[]
    B --> C[MQ]
    C --> D[]
    D --> E[ACK]
    E --> F[]
```

7. 分布式限流算法 ★★★★★

四种限流算法对比

令牌桶 VS 漏桶

```
flowchart LR
    subgraph
        A[] --> B[]
        B --> C[]
    end

    subgraph
        D[] --> E[]
        E --> F[/]
    end

    end
```

关键区别：

- 漏桶：匀速处理，不支持突发
- 令牌桶：桶满时允许一定程度的突发流量

8. 高频面试题 ★★★★★~★★★★★

Q1：分布式和微服务有什么区别？

参考答案：

- **分布式**：物理部署方式，解决单台机器性能扛不住的问题
- **微服务**：软件架构设计思想，解决系统代码堆积难以维护的问题
- **联系**：微服务架构天然需要分布式方式部署

Q2：如何解决分布式Session一致性问题？

参考答案：

1. **Session复制**：多台服务器间同步Session（浪费内存）
2. **Session集中存储**：存入Redis（推荐，性能高）
3. **Cookie存储**：安全性低、有大小限制
4. **Token机制**：使用JWT等Token代替Session（推荐，无状态）

Q3：什么是服务雪崩？如何解决？

参考答案：

服务雪崩：当系统中的一个服务故障，导致调用它的其他服务也相继故障，最终整个系统瘫痪。

解决方案：

Q4：一致性Hash算法了解吗？

参考答案：

解决分布式缓存数据倾斜问题

flowchart LR

A[Hash 0~2³²-1] --> B[Node A]

A --> C[Node B]

A --> D[Node C]

E[Key] --> |Hash| F[]

解决数据倾斜：引入虚拟节点，为每个真实节点分配多个虚拟节点

Q5：Zookeeper的Watch机制是什么原理？

参考答案：

1. 客户端可以在某个Znode上注册一个Watcher
2. 当该Znode的状态发生变化时
3. Zookeeper向客户端发送一条通知
4. **Watch只能被触发一次**，需要重新注册

Q6：如何实现分布式配置管理？

参考答案：

1. **Zookeeper**：存储在Znode中，利用Watch机制通知变更
2. **Apollo**：携程开源，支持配置变更实时推送、历史版本管理
3. **Nacos**：阿里巴巴开源，支持配置管理和服务发现
4. **Spring Cloud Config**：Spring Cloud生态的配置中心

Q7：Raft和ZAB协议有什么区别？

Q8：如何保证分布式系统的高可用？

参考答案：

1. **冗余备份**：多节点部署，避免单点故障
2. **故障检测与自动切换**：心跳机制，自动切换
3. **限流降级**：流量洪峰时自动降级非核心功能
4. **熔断机制**：快速失败，防止雪崩效应
5. **重试机制**：失败请求自动重试（需保证幂等性）

附录：分布式核心知识点星级

> 笔记说明：本笔记结合小林coding和黑马程序员整理，涵盖分布式核心面试知识点。建议面试前快速回顾。

Linux与Git命令面试笔记

Linux与Git命令核心面试知识点

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [Linux常用命令](#1-linux常用命令)
2. [进程与线程管理](#2-进程与线程管理)
3. [网络相关命令](#3-网络相关命令)
4. [Shell脚本](#4-shell脚本)
5. [Git命令](#5-git命令)

1. Linux常用命令 ★★☆☆

文件操作命令

权限管理命令

```
flowchart LR
    A[chmod] --> B[chmod 755 file]
    A --> C[]
    A --> D[r=4 w=2 x=1]
    E[chown] --> F[chown user:group file]
```

权限数字：

- `0` = 无权限
- `1` = 执行
- `2` = 写
- `4` = 读

示例：

```
chmod 755 file    # rwxr-xr-x
chmod 644 file    # rw-r--r--
chmod +x script   #
```

2. 进程与线程管理 ★★★

ps命令

```
ps aux    #
ps -ef    #
ps -T -p <pid>  #
```

ps显示内容：

top命令

```
top    #
top -p <pid>  #
top -H    #
```

top显示内容：

按1显示多CPU核心：

进程状态

杀进程命令

```
kill <pid>          #
kill -9 <pid>        #
kill -15 <pid>       #
pkill -f <>          #
```

查看负载情况

```
uptime              #
# Load Average: 1.00, 5.00, 10.00
# = 1515
```

负载判断：

- 负载值一般不超过CPU核数的1-1.5倍
- 超过1.5倍需要重视，会严重影响系统

3. 网络相关命令 ★★★

netstat命令

```
netstat -anp        #
netstat -tulpn       #
netstat -rn          #
```

netstat显示内容：

- TCP连接状态（LISTEN、ESTABLISHED等）
- 四元组信息（源IP、目标IP、源端口、目标端口）

查看端口占用

```
lsof -i:<>          #
netstat -tulpn | grep <>
```

端口连通性测试

```
telnet <IP> <>      #
nc -zv <IP> <>       # netcat
ping <IP>            #
```

ip和ifconfig

```
ip addr              # IP
ifconfig             #
```

4. Shell脚本 ★★★

文本处理三剑客

grep命令

```
grep ""              #
grep -r ""            #
grep -i ""            #
grep -n ""            #
grep -v ""            #
```

sed命令

```
#
sed -i 's///g'

#
sed -i 's/Hello/Hi/g' example.txt

# -i
# s///g
```

sed vs awk区别：

awk命令

```
#
awk '{print $1, $3}'      # 13
awk -F',' '{print $2}'    #

#
awk '{sum+=$3} END {print sum}'

# grep
grep "search_string" log_file | awk '{ print length }'
```

查找日志耗时最高的10条记录

```
# 10
sort -k3 -nr | head -n 10

# -k3
# -n
# -r
```

Shell脚本基础

```
#!/bin/bash
#
NAME="value"
#
if [ $a -gt $b ]; then
    echo "a > b"
fi
#
for i in {1..10}; do
    echo $i
done

---
```

5. Git命令 ★★★

Git工作原理

```
flowchart LR
    A[] -->|add| B[]
    B -->|commit| C[]
    C -->|push| D[]

    D -->|pull/fetch| C
    C -->|checkout| A
```

四个区域：

基础命令

```
#
git init                  #
git clone <url>           #

#
git add .                 #
git commit -m "message"  #
git push origin main      #
git pull origin main      #

#
git status                #
git diff                  #
git log                   #
git log --oneline         #
```

Git撤回操作

```
# git add
git reset HEAD <file>
```

```
# git commit
git reset --soft HEAD^

# git commit
git reset --mixed HEAD^

# git commit
git reset --hard HEAD^

# git push
git reset HEAD^
git push origin <branch> --force
```

Git分支管理

```
#
git branch <name>          #
git checkout <name>        #
git checkout -b <name>     #
git switch <name>          #

#
git merge <branch>         #
git rebase <branch>       #

#
git branch -d <name>       #
git branch -D <name>      #
```

git rebase vs git merge

Git冲突解决

```
# 1.
git status

# 2.
<<<<<<< HEAD

=====

>>>>>>> branch-name

# 3.

# 4.
git add <file>

# 5.
git commit -m "resolve conflict"
```

Git远程操作

```
#
git remote add origin <url>

#
git remote -v

#
git push -u origin <branch>

#
git branch --set-upstream-to=origin/<branch>
```

实用Git命令

```
#
git stash          #
git stash pop      #

#
git diff --staged  # vs
git diff HEAD~1    # vs
```

```
#
git checkout -- <file> #
git restore <file>     #

#
git blame <file>       #

---
```

高频面试题 ★★★~★★★★★

Q1：如何排查CPU跑到100%的问题？

参考答案：

1. 执行`top`命令，定位到占用CPU高的进程
2. 使用`ps -T -p <pid>`找到进程中占用比较高的线程
3. 使用`jstack <pid>`查看该线程的堆栈信息
4. 根据堆栈信息定位代码，看是否有死循环

Q2：Linux如何查看负载情况？

参考答案：

使用uptime或top命令查看Load Average：

- 三个数字依次是过去1分钟、5分钟、15分钟的平均负载
- 负载值一般不超过CPU核数的1-1.5倍
- 超过1.5倍需要重视，会影响系统性能

Q3：如何查看端口被哪个进程占用？

参考答案：

```
# 1lsof
lsof -i:<>

# 2netstat
netstat -tulpn | grep <>
```

Q4：如何测试端口是否可达？

参考答案：

```
# telnet
telnet <IP> <>

# nc
nc -zv <IP> <>
```

Q5：git rebase和merge有什么区别？

参考答案：

- ****merge****：创建一个新的合并提交，将两个分支的提交历史连接在一起，保留分支记录
- ****rebase****：将源分支上的每个提交复制到目标分支最新位置，使提交历史变成线性

Q6：如何撤回已经push的代码？

参考答案：

```
# 1.
git reset --hard HEAD^

# 2.
git push origin <branch> --force
```

Q7：Shell三剑客的区别？

参考答案：

附录：常用命令速查

```
ls cd mkdir rm cp mv cat head tail find
chmod chown
ps top kill pkill
netstat lsof telnet nc ping
grep sed awk wc
uptime df du free
Gitadd commit push pull merge rebase reset stash
```

>

笔记说明：本笔记结合小林coding和黑马程序员整理，涵盖Linux和Git命令核心面试知识点。建议面试前快速回顾。

Agent开发核心面试笔记

Agent开发核心面试知识点（Java版）

> 小林coding x 黑马程序员 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [AI Agent基础概念](#1-ai-agent基础概念)
2. [Spring AI框架详解](#2-spring-ai框架详解)
3. [Function Calling原理与实现](#3-function-calling原理与实现)
4. [Agent核心组件开发](#4-agent核心组件开发)
5. [主流Agent模式](#5-主流agent模式)
6. [Multi-Agent系统设计](#6-multi-agent系统设计)
7. [Memory与Context管理](#7-memory与context管理)
8. [RAG与Agent结合](#8-rag与agent结合)
9. [Agent安全与护栏](#9-agent安全与护栏)
10. [高频面试题](#10-高频面试题)

1. AI Agent基础概念 ★★★

什么是AI Agent？

AI Agent（智能体）是一种能够自主感知环境、做出决策并执行动作的智能系统。

```
flowchart LR
    A[LLM] --> B[]
    A --> C[]
    A --> D[]

    E[AI Agent] --> F[]
    E --> G[]
    E --> H[]
    E --> I[]
```

Agent四大核心能力

```
flowchart TD
    A[] --> B[Planning]
    A --> C[Memory]
    A --> D[Tools]
    A --> E[Action]

    B --> F[]
    B --> G[]
    C --> H[]
    C --> I[]
    D --> J[]
    D --> K[]
    E --> L[]
```

Agent工作流程

```
flowchart TD
    A[] --> B[]
    B --> C[]
    C --> D{}
    D --> E[]
    E --> F[]
    F --> G[]
    G --> H{?}
    H -->| | C
    H -->| | I[]
```

2. Spring AI框架详解 ★★★★★

2.1 Spring AI概述

Spring AI是Spring官方提供的AI应用开发框架，提供与AI模型交互的Spring友好型API。

> 官网：<https://spring.io/projects/spring-ai>

```
flowchart TB
    subgraph Spring_AI [Spring AI]
        A[Application] --> B[ChatClient]
        B --> C[Advisors]
        B --> D[Tools]
        B --> E[Memory]
        C --> F[ChatModel]
        D --> F
        E --> F
    end

    subgraph External [ ]
        G[OpenAI]
        H[Azure OpenAI]
        I[Anthropic Claude]
        J[ ]
        K[ ]
    end

    F --> G
    F --> H
    F --> I
    F --> J
    F --> K
```

2.2 核心组件

2.3 快速开始

Maven依赖：

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
</dependency>
```

配置文件：

```
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY}
    chat:
      options:
        model: gpt-4
```

基本使用：

```
@RestController
public class AIController {

    private final ChatClient chatClient;

    public AIController(ChatClient chatClient) {
        this.chatClient = chatClient;
    }

    @GetMapping("/chat")
    public String chat(@RequestParam String message) {
        return chatClient.prompt()
            .user(message)
            .call()
            .content();
    }
}
```

3. Function Calling原理与实现 ★★★★★

3.1 什么是Function Calling ?

Function Calling是LLM生成结构化工具调用的能力，让模型能调用外部工具获取实时信息或执行操作。

```
sequenceDiagram
    participant U as
    participant C as ChatClient
    participant LLM as LLM
    participant T as Tool

    U->>C:
    C->>LLM: ()
    LLM-->>C:
    C->>T:
    T-->>C:
    C->>LLM:
    LLM-->>C:
    C->>U:
```

3.2 Spring AI实现Function Calling

方式一：@Tool注解（推荐）

```
/**
 * - @Tool
 */
public class WeatherTools {

    @Tool(description = "")
    public String getWeather(@ToolParam(description = "") String city) {
        // API
        return "";
    }

    @Tool(description = "")
    public String getDateTime() {
        return LocalDateTime.now().toString();
    }
}

// ChatClient
@Service
public class WeatherService {

    private final ChatClient chatClient;

    public WeatherService(ChatClient.Builder builder) {
        this.chatClient = builder
            .defaultTools(new WeatherTools()) //
            .build();
    }

    public String ask(String question) {
        return chatClient.prompt()
            .user(question)
            .call()
            .content();
    }
}
```

方式二：FunctionToolCallback（灵活控制）

```
@Configuration
public class ToolConfiguration {

    @Bean
    public ToolCallback weatherToolCallback() {
        return FunctionToolCallback.builder("getWeather", new WeatherService())
            .description("")
            .inputType(WeatherRequest.class)
            .build();
    }
}
```

```

@Bean
public ToolCallback calculatorToolCallback() {
    return FunctionToolCallback.builder("calculate", new CalculatorService())
        .description("")
        .inputType(CalculatorRequest.class)
        .build();
}

//
@Service
public class AgentService {

    private final ChatClient chatClient;

    public AgentService(ChatClient.Builder builder, ToolCallback weatherTool) {
        this.chatClient = builder
            .defaultToolCallbacks(weatherTool)
            .build();
    }
}

```

方式三：@Bean + @Description声明式

```

@Configuration
public class FunctionConfiguration {

    @Bean
    @Description("")
    public Function<WeatherRequest, WeatherResponse> weatherFunction() {
        return new WeatherService();
    }

    @Bean
    @Description("")
    public Function<CalculatorRequest, CalculatorResponse> calculateFunction() {
        return new CalculatorService();
    }
}

// bean
public String ask(String question) {
    return chatClient.prompt()
        .user(question)
        .functions("weatherFunction", "calculateFunction") // bean
        .call()
        .content();
}

```

3.3 工具接口定义

```

//
public record WeatherRequest(String city, String unit) {}

//
public record WeatherResponse(double temperature, String condition) {}

//
public class WeatherService implements Function<WeatherRequest, WeatherResponse> {

    @Override
    public WeatherResponse apply(WeatherRequest request) {
        // API
        return new WeatherResponse(22.5, "");
    }
}

```

3.4 Tool Schema完整定义

```

// Tool
public class AdvancedToolDefinition {

    // @Tool
    @Tool(
        description = "",

```

```

        name = "searchFlights"
    )
    public FlightResponse searchFlights(
        @ToolParam(description = "", required = true) String origin,
        @ToolParam(description = "", required = true) String destination,
        @ToolParam(description = "") String date,
        @ToolParam(description = "", allowedValues = {"one_way", "round_trip"})
            String tripType
    ) {
        //
        return new FlightResponse();
    }

    // Function
    public class FlightSearchService implements Function<FlightRequest, FlightResponse> {

        public record FlightRequest(
            String origin,
            String destination,
            String date,
            String tripType
        ) {}

        public record FlightResponse(
            List<String> flights,
            double lowestPrice
        ) {}

        @Override
        public FlightResponse apply(FlightRequest request) {
            //
            return new FlightResponse(List.of("CA1234", "MU5678"), 850.0);
        }
    }
}

```

3.5 工具调用策略

```

//
List<ToolCallback> tools = List.of(
    weatherTool,
    calculatorTool,
    searchTool
);

chatClient.prompt()
    .user(" 100 * 50")
    .toolCallbacks(tools) //
    .call()
    .content();

```

4. Agent核心组件开发 ★★★★★

4.1 ChatClient详解

```

flowchart TB
    A[Prompt] --> B[Advisors]
    B --> C[Tools]
    C --> D[ChatModel]
    D --> E[LLM]
    E --> F[Response]
    F --> G[Advisors]
    G --> H[Result]

```

ChatClient使用模式：

```

// Builder
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(new SimpleLoggerAdvisor())
    .defaultTools(weatherTools)
    .build();

// create

```

```

ChatClient chatClient = ChatClient.create(chatModel);

// ChatClient.Builder
@Service
public class MyService {

    public MyService(ChatClient.Builder builder) {
        ChatClient client = builder
            .defaultTools(myTools)
            .build();
    }
}

```

4.2 Advisor拦截器链

Advisor是Spring AI的核心扩展机制，类似于Spring的拦截器或AOP切面。

```

// Advisor
ChatClient chatClient = ChatClient.builder(chatModel)
    .defaultAdvisors(
        //
        new SimpleLoggerAdvisor(),

        //
        MessageChatMemoryAdvisor.builder(chatMemory)
            .conversationId("user-123")
            .build(),

        //
        SafeGuardAdvisor.builder()
            .sensitiveWords(List.of("password", "secret"))
            .build()
    )
    .build();

```

自定义Advisor：

```

@Component
public class CustomAdvisor implements CallAdvisor {

    @Override
    public String getName() {
        return "customAdvisor";
    }

    @Override
    public int getOrder() {
        return 0; //
    }

    @Override
    public ChatClientResponse adviseCall(
        ChatClientRequest request,
        CallAdvisorChain chain) {

        //
        System.out.println(": " + request);

        //
        ChatClientResponse response = chain.nextCall(request);

        //
        System.out.println(": " + response);

        return response;
    }
}

```

Advisor执行顺序：

```

: Advisor1 → Advisor2 → Advisor3 → ChatModel
: ChatModel → Advisor3 → Advisor2 → Advisor1

```

4.3 ReAct模式实现

ReAct = Reasoning + Acting，Spring AI通过ReActAgent实现：

```
// Spring AI Agent UtilsReAct Agent
@Configuration
public class ReActAgentConfig {

    @Bean
    public ChatClient reactChatClient(ChatModel chatModel) {
        return ChatClient.builder(chatModel)
            .defaultTools(new WeatherTools(), new SearchTools())
            .build();
    }
}

// AdvisorReAct
@Service
public class ReActAgentService {

    private final ChatClient chatClient;

    public ReActAgentService(ChatClient.Builder builder) {
        this.chatClient = builder
            .defaultTools(new CalculatorTools())
            .build();
    }

    public String execute(String task) {
        String result = chatClient.prompt()
            .user(task)
            .call()
            .content();
        return result;
    }
}
```

4.4 Chain of Thought (CoT)

```
// Zero-shot CoT
String response = chatClient.prompt()
    .user("""
        : 354

        """)
    .call()
    .content();

// Few-shot CoT
String response = chatClient.prompt()
    .user("""
        Q: 240
        A: ...
        22*2=42...
        40-2=3838

        Q: 3320
        A:
        """)
    .call()
    .content();
```

4.5 Evaluator-Optimizer模式

```
//
@Service
public class EvaluatorOptimizerService {

    private final ChatClient chatClient;

    public EvaluatorOptimizerService(ChatClient chatClient) {
        this.chatClient = chatClient;
    }

    public String optimize(String task) {
        String solution = chatClient.prompt()
            .user(": " + task)
            .call()
            .content();
    }
}
```

```

        //
        String evaluation = chatClient.prompt()
            .user(": " + solution)
            .call()
            .content();

        //
        if (!isSatisfactory(evaluation)) {
            return optimize(task); //
        }

        return solution;
    }
}

```

5. 主流Agent模式 ★★★★★

5.1 ReAct模式（必考）

ReAct = Reasoning + Acting，推理与行动交替进行

```

flowchart TD
    A[] --> B[Thought: ]
    B --> C[Action: ]
    C --> D[Observation: ]
    D --> E{?}
    E --> B
    E --> F[Response]

    subgraph ReAct
        B --> C --> D --> E
    end
end

```

ReAct完整示例：

```

@Service
public class ReActWeatherAgent {

    private final ChatClient chatClient;

    public ReActWeatherAgent(ChatClient.Builder builder) {
        this.chatClient = builder
            .defaultTools(new WeatherTools())
            .build();
    }

    /**
     * ReAct
     *
     * Thought
     * ActiongetWeather
     * Action Input{"city": ""}
     * Observation25
     * Response25
     */
    public String ask(String question) {
        return chatClient.prompt()
            .user(question)
            .call()
            .content();
    }
}

```

5.2 Plan-and-Execute模式

```

@Service
public class PlanExecuteAgent {

    private final ChatClient chatClient;

    // 1. -
    public List<String> plan(String task) {
        return chatClient.prompt()

```

```

        .user(": " + task)
        .call()
        .entity(new TypeReference<>() {});
    }

    // 2. -
    public String execute(String task) {
        List<String> steps = plan(task);
        String result = "";

        for (String step : steps) {
            String stepResult = chatClient.prompt()
                .user(": " + step + "\n : " + result)
                .call()
                .content();
            result = stepResult;
        }

        return result;
    }
}

```

5.3 自定义Agent实现

```

@Component
public class CustomAgent {

    private final ChatClient chatClient;
    private final ChatMemory chatMemory;

    public CustomAgent(ChatClient.Builder builder, ChatMemory chatMemory) {
        this.chatClient = builder
            .defaultAdvisors(
                MessageChatMemoryAdvisor.builder(chatMemory)
                    .conversationId("agent-1")
                    .build()
            )
            .defaultTools(new SearchTools(), new CalculatorTools())
            .build();
        this.chatMemory = chatMemory;
    }

    public String run(String userInput) {
        // 1.
        String plan = chatClient.prompt()
            .user(": " + userInput)
            .call()
            .content();

        // 2.
        String result = chatClient.prompt()
            .user(": " + plan)
            .call()
            .content();

        // 3.
        String reflection = chatClient.prompt()
            .user(": " + result)
            .call()
            .content();

        return reflection;
    }

    //
    public void clearMemory() {
        chatMemory.clear("agent-1");
    }
}

```

5.4 多轮对话Agent

```

@Configuration
public class ConversationalAgentConfig {

    @Bean

```

```

public ChatMemory chatMemory() {
    return new InMemoryChatMemory();
}

@Bean
public ChatClient conversationalChatClient(
    ChatModel chatModel,
    ChatMemory chatMemory) {

    return ChatClient.builder(chatModel)
        .defaultAdvisors(
            //
            new SimpleLoggerAdvisor(),
            //
            MessageChatMemoryAdvisor.builder(chatMemory)
                .conversationId("user-session")
                .build()
        )
        .defaultTools(new WeatherTools())
        .build();
}

//
@RestController
public class ChatController {

    private final ChatClient chatClient;

    public ChatController(ChatClient conversationalChatClient) {
        this.chatClient = conversationalChatClient;
    }

    @PostMapping("/chat")
    public String chat(@RequestBody ChatRequest request) {
        return chatClient.prompt()
            .user(request.message())
            .call()
            .content();
    }
}

```

6. Multi-Agent系统设计 ★★★★★

6.1 Spring AI Multi-Agent生态

```

flowchart TB
    subgraph Spring_AI [Spring AI]
        A[Spring AI Core]
        B[Spring AI Alibaba]
        C[Agent Utils]
    end

    subgraph Spring_AI_Alibaba [Spring AI Alibaba]
        D[SequentialAgent]
        E[ParallelAgent]
        F[LlmRoutingAgent]
        G[StateGraph]
    end

```

6.2 SequentialAgent（顺序执行）

```

// Maven
/*
<dependency>
    <groupId>com.alibaba.cloud.ai</groupId>
    <artifactId>spring-ai-alibaba</artifactId>
</dependency>
*/

@Configuration
public class SequentialAgentConfig {

```



```

@Bean
public SequentialAgent contentPipeline(ChatModel chatModel) {
    // Agent
    ReactAgent writerAgent = ReactAgent.builder()
        .name("writer_agent")
        .model(chatModel)
        .instruction(": {input}")
        .outputKey("article")
        .build();

    // Agent
    ReactAgent reviewerAgent = ReactAgent.builder()
        .name("reviewer_agent")
        .model(chatModel)
        .instruction(": {article}")
        .outputKey("reviewed_article")
        .build();

    // Agent
    ReactAgent translatorAgent = ReactAgent.builder()
        .name("translator_agent")
        .model(chatModel)
        .instruction(": {reviewed_article}")
        .outputKey("final_article")
        .build();

    return SequentialAgent.builder()
        .name("content_pipeline")
        .subAgents(List.of(writerAgent, reviewerAgent, translatorAgent))
        .build();
}

//
@Service
public class ContentService {

    private final SequentialAgent contentPipeline;

    public ContentService(SequentialAgent contentPipeline) {
        this.contentPipeline = contentPipeline;
    }

    public String createContent(String topic) {
        Optional<OverAllState> result = contentPipeline.invoke(topic);

        return result
            .map(state -> state.value("final_article").orElse(""))
            .orElse("");
    }
}

```

6.3 ParallelAgent (并行执行)

```

@Configuration
public class ParallelAgentConfig {

    @Bean
    public ParallelAgent creativeParallelAgent(ChatModel chatModel) {

        ReactAgent proseAgent = ReactAgent.builder()
            .name("prose_writer")
            .model(chatModel)
            .instruction(": {input}")
            .outputKey("prose_result")
            .build();

        ReactAgent poetryAgent = ReactAgent.builder()
            .name("poetry_writer")
            .model(chatModel)
            .instruction(": {input}")
            .outputKey("poetry_result")
            .build();

        ReactAgent summaryAgent = ReactAgent.builder()
            .name("summarizer")

```

```

        .model(chatModel)
        .instruction(": {input}")
        .outputKey("summary_result")
        .build();

    return ParallelAgent.builder()
        .name("creative_parallel")
        .subAgents(List.of(proseAgent, poetryAgent, summaryAgent))
        .mergeOutputKey("merged_results")
        .build();
    }
}

//
@Service
public class CreativeService {

    private final ParallelAgent parallelAgent;

    public String generate(String topic) {
        Optional<OverAllState> result = parallelAgent.invoke(topic);

        return result
            .map(state -> {
                StringBuilder sb = new StringBuilder();
                state.value("prose_result").ifPresent(v -> sb.append(": ").append(v).append("\n"));
                state.value("poetry_result").ifPresent(v -> sb.append(": ").append(v).append("\n"));
                state.value("summary_result").ifPresent(v -> sb.append(": ").append(v));
                return sb.toString();
            })
            .orElse("");
    }
}

```

6.4 LlmRoutingAgent (智能路由)

```

@Configuration
public class RoutingAgentConfig {

    @Bean
    public LlmRoutingAgent smartRouter(ChatModel chatModel) {

        ReactAgent technicalAgent = ReactAgent.builder()
            .name("technical_expert")
            .model(chatModel)
            .description("")
            .instruction("")
            .outputKey("technical_output")
            .build();

        ReactAgent businessAgent = ReactAgent.builder()
            .name("business_analyst")
            .model(chatModel)
            .description("")
            .instruction("")
            .outputKey("business_output")
            .build();

        ReactAgent creativeAgent = ReactAgent.builder()
            .name("creative_writer")
            .model(chatModel)
            .description("")
            .instruction("")
            .outputKey("creative_output")
            .build();

        return LlmRoutingAgent.builder()
            .name("smart_router")
            .model(chatModel)
            .subAgents(List.of(technicalAgent, businessAgent, creativeAgent))
            .build();
    }
}

// - LLMAgent
@Service

```

```

public class RoutingService {

    private final LlmRoutingAgent router;

    public String route(String question) {
        // Agent
        return router.invoke(question).toString();
    }
}

```

6.5 StateGraph（状态机 workflow）

```

@Configuration
public class StateGraphWorkflowConfig {

    @Bean
    public CompiledGraph workflowGraph(ChatModel chatModel) {

        // key
        KeyStrategyFactory keyFactory = () -> {
            Map<String, KeyStrategy> strategies = new HashMap<>();
            strategies.put("input", new ReplaceStrategy());
            strategies.put("processed", new ReplaceStrategy());
            strategies.put("result", new ReplaceStrategy());
            return strategies;
        };

        //
        StateGraph workflow = new StateGraph(keyFactory)
        //
        .addNode("input_handler", node_async(state -> {
            String input = state.value("input", "").toString();
            return Map.of("processed", input.toUpperCase());
        }))
        .addNode("validator", node_async(state -> {
            String processed = state.value("processed", "").toString();
            boolean valid = processed.length() > 5;
            return Map.of("result", valid ? "VALID" : "INVALID");
        }))
        .addNode("processor", node_async(state ->
            Map.of("result", "Processed: " + state.value("processed", "").toString())
        ))
        .addNode("error_handler", node_async(state ->
            Map.of("result", "Error: input too short")
        ));

        //
        workflow.addEdge(START, "input_handler");
        workflow.addEdge("input_handler", "validator");

        //
        workflow.addConditionalEdges(
            "validator",
            edge_async(state -> {
                String result = state.value("result", "").toString();
                return result; // "VALID" "INVALID"
            }),
            Map.of(
                "VALID", "processor",
                "INVALID", "error_handler"
            )
        );

        workflow.addEdge("processor", END);
        workflow.addEdge("error_handler", END);

        return workflow.compile();
    }
}

//
@Service
public class WorkflowService {

    private final CompiledGraph workflow;
}

```

```

    public String execute(String input) {
        return workflow.invoke(Map.of("input", input))
            .map(state -> state.value("result").orElse("").toString())
            .orElse("");
    }
}

```

6.6 Multi-Agent通信模式

```

sequenceDiagram
    participant U as User
    participant O as Orchestrator
    participant A1 as Agent1
    participant A2 as Agent2

    U->>O:
    O->>A1: 1
    O->>A2: 2

    par
        A1->>A1: 1
        A2->>A2: 2
    end

    A1-->>O: 1
    A2-->>O: 2

    O->>O:
    O-->>U:

```

7. Memory与Context管理 ★★★★★

7.1 ChatMemory架构

```

flowchart TB
    subgraph ChatMemory
        A[MessageHistory] --> B[InMemoryChatMemory]
        A --> C[RedisChatMemory]
        A --> D[JdbcChatMemory]
    end

    subgraph Message
        E[UserMessage]
        F[AssistantMessage]
        G[SystemMessage]
    end

```

7.2 对话记忆实现

```

//
@Configuration
public class MemoryConfig {

    @Bean
    public ChatMemory chatMemory() {
        //
        return new InMemoryChatMemory();

        // Redis
        // return new RedisChatMemory(connectionFactory);

        //
        // return new JdbcChatMemory(dataSource);
    }

    @Bean
    public ChatClient chatClient(ChatModel chatModel, ChatMemory chatMemory) {
        return ChatClient.builder(chatModel)
            .defaultAdvisors(
                MessageChatMemoryAdvisor.builder(chatMemory)
                    .conversationId("default-session")
                    .build()
            )
    }
}

```

```

        .build();
    }
}

//
@Service
public class ConversationalService {

    private final ChatMemory chatMemory;
    private final ChatClient chatClient;

    public ConversationalService(ChatMemory chatMemory, ChatClient.Builder builder) {
        this.chatMemory = chatMemory;
        this.chatClient = builder
            .defaultAdvisors(
                MessageChatMemoryAdvisor.builder(chatMemory)
                    .conversationId("user-123")
                    .build()
            )
            .build();
    }

    public String chat(String message) {
        return chatClient.prompt()
            .user(message)
            .call()
            .content();
    }

    //
    public void clearHistory() {
        chatMemory.clear("user-123");
    }
}

```

7.3 多级记忆系统

```

@Service
public class MultiLevelMemoryService {

    //
    private final ChatMemory shortTermMemory;

    //
    private final VectorStore vectorStore;

    public MultiLevelMemoryService(
        ChatMemory shortTermMemory,
        VectorStore vectorStore) {
        this.shortTermMemory = shortTermMemory;
        this.vectorStore = vectorStore;
    }

    //
    public void storeToLongTerm(String content) {
        vectorStore.add(List.of(new Document(content)));
    }

    //
    public String retrieveFromLongTerm(String query) {
        List<Document> docs = vectorStore.similaritySearch(query);
        return docs.stream()
            .map(Document::getContent)
            .collect(Collectors.joining("\n"));
    }

    //
    public String chatWithMemory(String query) {
        // 1.
        String longTermContext = retrieveFromLongTerm(query);

        // 2.
        String context = ":\n" + longTermContext + "\n\n:\n" + query;

        // 3.
        return shortTermMemory.chat(context);
    }
}

```

```

    }
}

```

7.4 Context Window管理

```

//
@Service
public class SummarizingMemoryService {

    private final ChatMemory chatMemory;
    private final ChatClient chatClient;

    public SummarizingMemoryService(
        ChatMemory chatMemory,
        ChatClient.Builder builder) {
        this.chatMemory = chatMemory;
        this.chatClient = builder.build();
    }

    //
    public void summarizeIfNeeded(String conversationId, int maxMessages) {
        var messages = chatMemory.get(conversationId);

        if (messages.size() > maxMessages) {
            //
            String summary = chatClient.prompt()
                .user(": " + messages)
                .call()
                .content();

            //
            chatMemory.clear(conversationId);
            chatMemory.add(conversationId, new AssistantMessage(": " + summary));
        }
    }
}

```

8. RAG与Agent结合 ★★★★★

8.1 RAG-Agent架构

```

flowchart LR
    A[Query] --> B[Agent]
    B --> C[Retriever]
    C --> D[]
    D --> E[]
    E --> F[]
    F --> G[LLM]
    G --> H[]

```

8.2 RAG实现

```

@Configuration
public class RagConfig {

    @Bean
    public VectorStore vectorStore() {
        // Pinecone
        // return new PineconeVectorStore(...);

        //
        return new SimpleVectorStore();
    }

    @Bean
    public ChatClient ragChatClient(
        ChatModel chatModel,
        VectorStore vectorStore) {

        return ChatClient.builder(chatModel)
            .defaultAdvisors(
                // RAGAdvisor
                new RetrievalAugmentedAdvisor(

```

```

        vectorStore,
        AdvisorUtils.defaultConverter()
    )
    )
    .build();
}

//
@Service
public class RagService {

    private final VectorStore vectorStore;
    private final ChatClient chatClient;

    public RagService(VectorStore vectorStore, ChatClient.Builder builder) {
        this.vectorStore = vectorStore;
        this.chatClient = builder.build();
    }

    //
    public void indexDocument(String content) {
        vectorStore.add(List.of(new Document(content)));
    }

    // RAG
    public String query(String question) {
        return chatClient.prompt()
            .user(question)
            .call()
            .content();
    }
}

```

8.3 混合检索RAG

```

@Service
public class HybridRagService {

    private final VectorStore vectorStore;
    private final ChatClient chatClient;

    public HybridRagService(
        VectorStore vectorStore,
        ChatClient.Builder builder) {
        this.vectorStore = vectorStore;
        this.chatClient = builder.build();
    }

    public String hybridQuery(String question) {
        // 1.
        List<Document> semanticResults = vectorStore.similaritySearch(question);

        // 2.
        List<Document> keywordResults = keywordSearch(question);

        // 3. RRF
        List<Document> fusedResults = reciprocalRankFusion(
            semanticResults,
            keywordResults
        );

        // 4.
        String context = fusedResults.stream()
            .map(Document::getContent)
            .collect(Collectors.joining("\n"));

        // 5.
        return chatClient.prompt()
            .user(":\n" + context + "\n\n:" + question)
            .call()
            .content();
    }
}

```

9. Agent安全与护栏 ★★★★★

9.1 安全威胁类型

```
flowchart TB
    A[Agent] --> B[]
    A --> C[]
    A --> D[]
    A --> E[]

    B --> B1[]
    C --> C1[]
    D --> D1[]
    E --> E1[]
```

9.2 SafeGuardAdvisor

```
@Configuration
public class SecurityConfig {

    @Bean
    public ChatClient secureChatClient(ChatModel chatModel) {
        return ChatClient.builder(chatModel)
            .defaultAdvisors(
                //
                SafeGuardAdvisor.builder()
                    .sensitiveWords(List.of(
                        "password",
                        "secret",
                        "api-key",
                        "token"
                    ))
                    .build(),
                //
                ContentFilterAdvisor.builder()
                    .blockedPatterns(List.of("", ""))
                    .build()
            )
            .build();
    }
}
```

9.3 自定义安全护栏

```
@Component
public class CustomSecurityAdvisor implements CallAdvisor {

    private final List<String> blockedCommands = List.of(
        "rm -rf",
        "drop table",
        "delete from"
    );

    private final List<String> sensitivePatterns = List.of(
        "\\d{15,18}", //
        "\\d{3,4}-\\d{7,8}", //
        "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.\\.[a-zA-Z]{2,}" //
    );

    @Override
    public ChatClientResponse adviseCall(
        ChatClientRequest request,
        CallAdvisorChain chain) {

        // 1.
        String userInput = request.prompt().getUserMessage().getContent();
        validateInput(userInput);

        // 2.
        ChatClientResponse response = chain.nextCall(request);

        // 3.
```



```

        String output = response.getResult().getOutput().getText();
        validateOutput(output);

        return response;
    }

    private void validateInput(String input) {
        //
        for (String cmd : blockedCommands) {
            if (input.contains(cmd)) {
                throw new SecurityException(": " + cmd);
            }
        }
    }

    private void validateOutput(String output) {
        //
        for (String pattern : sensitivePatterns) {
            if (Pattern.matches(pattern, output)) {
                //
                throw new SecurityException("");
            }
        }
    }
}

```

9.4 MCP安全机制

```

// MCP OAuthSpring AI MCP
@Configuration
public class McpSecurityConfig {

    @Bean
    public TokenVerifier tokenVerifier() {
        return new CustomTokenVerifier();
    }

    public class CustomTokenVerifier implements TokenVerifier {
        @Override
        public AccessToken verify(String token) {
            // JWT
            try {
                Jwt jwt = JwtParser()
                    .verifyWith(secretKey)
                    .parse(token);

                return new AccessToken(
                    jwt.getSubject(),
                    List.of("read", "execute"),
                    jwt.getExpiration()
                );
            } catch (JwtException e) {
                return null;
            }
        }
    }
}

```

10. 高频面试题 ★★★~★★★★★

Q1：什么是AI Agent？和传统LLM有什么区别？

参考答案：

- AI Agent是能够自主感知环境、做出决策并执行动作的智能系统
- 传统LLM是被动问答，单次交互无状态
- Agent具有**规划、记忆、工具调用、环境交互**四大核心能力
- Spring AI通过`ChatClient` + `@Tool` + `Advisor`实现Agent

Q2：Spring AI如何实现Function Calling？

参考答案：

```
// @Tool
public class WeatherTools {
    @Tool(description = "")
    public String getWeather(@ToolParam(description = "") String city) {
        return "";
    }
}

//
chatClient.prompt()
    .user("")
    .tools(new WeatherTools())
    .call()
    .content();

// FunctionToolCallback
ToolCallback callback = FunctionToolCallback.builder("getWeather", new WeatherService())
    .description("")
    .inputType(WeatherRequest.class)
    .build();
```

Q3：ReAct模式的原理是什么？

参考答案：

- ReAct = Reasoning + Acting，推理与行动交替进行
- 格式：`Thought → Action → Observation → ... → Response`
- Spring AI中通过`ChatClient` + 工具自动实现ReAct循环
- 模型思考是否需要调用工具，调用后观察结果，继续推理直到完成

Q4：Spring AI的Advisor是什么？执行顺序？

参考答案：

- Advisor是拦截器链，类似Spring AOP的切面
- 用于日志、记忆、安全、工具等功能扩展
- 执行顺序：按`getOrder()`值从小到大执行
- 请求阶段：Advisor1 → Advisor2 → ChatModel
- 响应阶段：ChatModel → Advisor2 → Advisor1

Q5：Spring AI Alibaba的Multi-Agent有哪些？如何选型？

参考答案：

Q6：如何保证Agent的可靠性？

参考答案：

1. **状态机控制**：使用StateGraph精确控制 workflow
2. **错误处理**：工具调用失败时重试或回退
3. **验证机制**：执行前验证参数，执行后检查结果
4. **Human-in-the-loop**：关键决策需要人工确认
5. **监控告警**：使用`SimpleLoggerAdvisor`记录日志

Q7：Spring AI如何实现对话记忆？

参考答案：

```
// ChatMemory
@Bean
public ChatMemory chatMemory() {
    return new InMemoryChatMemory();
}

@Bean
public ChatClient chatClient(ChatModel chatModel, ChatMemory chatMemory) {
    return ChatClient.builder(chatModel)
        .defaultAdvisors(
            MessageChatMemoryAdvisor.builder(chatMemory)
                .conversationId("user-123")
                .build()
        )
        .build();
}
```

}

Q8：RAG和Agent如何结合？

参考答案：

- 使用`RetrievalAugmentedAdvisor`实现RAG
- Agent判断是否需要检索知识
- 从向量数据库检索相关文档
- 检索结果作为上下文组装到Prompt
- LLM生成回答

Q9：如何设计企业级AI客服Agent？

参考答案：

1. **架构设计**：
 - `LlmRoutingAgent`：意图识别，路由到不同处理Agent
 - `SequentialAgent`：处理→审核→回复
 - `ParallelAgent`：并行处理多个相关问题
2. **核心能力**：
 - RAG：检索企业知识库
 - 工具调用：CRM、工单系统
 - 记忆：用户画像、会话历史
3. **安全保证**：
 - `SafeGuardAdvisor`：敏感词过滤
 - `ContentFilterAdvisor`：内容审核
 - 审计日志

Q10：Spring AI Agent和LangChain如何选型？

参考答案：

附录：Spring AI Agent知识点星级

参考资料

- [Spring AI官方文档](<https://spring.io/projects/spring-ai>)
- [Spring AI API Reference](<https://docs.spring.io/spring-ai/reference/api>)
- [Spring AI Alibaba](<https://github.com/alibaba/spring-ai-alibaba>)
- [Spring AI Agent Utils](<https://github.com/spring-ai-community/spring-ai-agent-utils>)
- [MCP Python SDK](<https://github.com/modelcontextprotocol/python-sdk>)

> 笔记说明：本笔记以Java/Spring AI为核心，涵盖AI Agent开发核心知识点。包括Spring AI框架、Function Calling实现、@Tool注解、Advisor机制、ReAct模式、Multi-Agent设计（Sequential/Parallel/Routing/StateGraph）、Memory管理、RAG结合、安全护栏等。建议Java开发者面试前深入理解。

JVM深入理解笔记

JVM深入理解面试笔记

> 黑马程序员Java面试专题 + 小林coding面试题补充 | 难度星级：★~★★★★★ | 面试频率：高/中/低

目录

1. [JVM体系结构](#1-jvm体系结构)
2. [内存结构详解](#2-内存结构详解)
3. [堆内存详解](#3-堆内存详解)
4. [垃圾回收算法](#4-垃圾回收算法)
5. [垃圾收集器](#5-垃圾收集器)
6. [类加载机制](#6-类加载机制) | [6.6 SPI机制](#66-spi机制打破双亲委派的典型场景)
7. [运行时常量池](#7-运行时常量池)
8. [字节码执行](#8-字节码执行)
9. [调优参数与工具](#9-调优参数与工具)
10. [高频面试题](#10-高频面试题)

1. JVM体系结构

1.1 JVM整体架构 ★★☆☆☆

```
flowchart TB
    subgraph A["\nClassLoader"]
    end

    subgraph B["\nMethod Area"]
    C["\nHeap"]
    D["Java\nVM Stack"]
    E["\nNative Stack"]
    F["\nPC Register"]
    end

    subgraph G["\nInterpreter"]
    H["JIT\nJIT Compiler"]
    I["\nGC"]
    end

    subgraph J["\nJNI"]
    K["\nNative Libraries"]
    end

    A -->|.class| B
    G -->| | D
    H -->| | G
    I -->| | C
    J --> K
```

1.2 线程私有 vs 线程共享 ★★☆☆☆

2. 内存结构详解

2.1 栈帧结构 ★★★★★☆

```
flowchart TB
```

```

subgraph Stack Frame
    A[\nLocal Variables]
    B[\nOperand Stack]
    C[\nDynamic Linking]
    D[\nReturn Address]
end

subgraph
    E[] -->|| A
    A -->|| F[]
end

```

栈帧组成：

2.2 常见异常 ★★☆☆☆

2.3 JVM内存模型详细介绍 ★★★★★☆

> 面试官：JVM的内存模型介绍一下

>

> 候选人：根据 JDK 8 规范，JVM 运行时内存共分为以下几个部分：

>

> 1. 程序计数器：可以看作是当前线程所执行的字节码的行号指示器，用于存储当前线程正在执行的 Java 方法的 JVM 指令地址。如果线程执行的是 Native 方法，计数器值为 null。是唯一一个在 Java 虚拟机规范中没有规定任何 OutOfMemoryError 情况的区域。

>

> 2. Java 虚拟机栈：每个线程都有自己独立的 Java 虚拟机栈，生命周期与线程相同。每个方法在执行时都会创建一个栈帧。

>

> 3. 本地方法栈：与 Java 虚拟机栈类似，主要为虚拟机使用到的 Native 方法服务，在 HotSpot 虚拟机中和 Java 虚拟机栈合二为一。

>

> 4. Java 堆：是 JVM 中最大的一块内存区域，被所有线程共享。用于存放对象实例。从内存回收角度，堆被划分为新生代和老年代。

>

> 5. 方法区（元空间）：在 JDK 1.8 及以后的版本中，方法区被元空间取代，使用本地内存。用于存储已被虚拟机加载的类信息、常量、静态变量等数据。

>

> 6. 运行时常量池：是方法区的一部分，用于存放编译期生成的各种字面量和符号引用。

2.4 堆和栈的区别 ★★★★★☆

> 面试官：JVM内存模型里的堆和栈有什么区别？

>

> 候选人：

> - 用途：栈主要用于存储局部变量、方法调用的参数、方法返回地址；堆用于存储对象的实例

> - 生命周期：栈中的数据当方法调用结束时随之消失；堆中的对象生命周期不确定，通过GC管理

> - 存取速度：栈的存取速度比堆快；堆的存取速度相对较慢

> - 存储空间：栈的空间相对较小，固定大小；堆的空间较大，动态扩展

> - 可见性：栈中的数据对线程是私有的；堆中的数据对线程是共享的

2.5 堆分为哪几部分 ★★★★★★（高频）

> 面试官：堆分为哪几部分呢？

>

> 候选人：Java堆主要分为以下几个部分：

>

> 1. 新生代（Young Generation）：

> - Eden区：大多数新创建的对象首先存放在这里

- > - Survivor Spaces：分为S0和S1两个区域，用于在每次Minor GC后存放存活对象
- >
- > 2. 老年代（Old Generation/Tenured Generation）：存放过一次或多次Minor GC仍存活的对象
- >
- > 3. 元空间（Metaspace）：从Java 8开始，替代了永久代，用于存储类的元数据信息
- >
- > 4. 大对象区（Humongous Objects）：在G1垃圾收集器中，为大对象分配专门的区域

2.6 大对象分配 ★★★☆☆

- > 面试官：如果有个大对象一般是在哪个区域？
- >
- > 候选人：大对象通常会直接分配到老年代。
- >
- > 原因：
- > - 新生代空间相对较小，如果将大对象分配到新生代，可能会很快导致Minor GC频繁
- > - 大对象需要连续的内存空间，老年代空间更大，更适合存储大对象
- > - 减少因内存碎片导致的分配失败

3. 堆内存详解

3.1 堆内存结构 ★★★★★（高频）

```

flowchart TB
    subgraph Heap
        subgraph Young_Generation
            A[Eden\n8/10]
            B[From Survivor\n1/10]
            C[To Survivor\n1/10]
        end
        subgraph Old_Generation
            D[Tenured\n2/3]
        end
    end

    A -->|GC| B
    B -->|| D
    C -->|GC| B
  
```

默认比例：Eden：Survivor = 8：1：1

3.2 对象分配与晋升 ★★★★★☆

```

flowchart TD
    A[] --> B{Eden\n?}
    B -->|| C[Eden]
    B -->|| D[Minor GC]

    C --> E{Minor GC\nEden?}
    E -->|,<| F[Survivor]
    E -->|,>=| G[]
    E -->|| H[]

    F --> I[+1]
    I --> J{>=15?}
    J -->|| K[Survivor]
    J -->|| G
  
```

对象年龄阈值：默认15岁可晋升老年代

4. 垃圾回收算法

4.1 四种算法对比 ★★★★★（高频）

```

flowchart LR
  
```

```

subgraph
    A1[] --> A2[]
    A1 --> A3[]
    A2 --> A4[\n]
end

subgraph
    B1[] --> B2[From]
    B2 --> B3[GCTo]
    B3 --> B4[From]
end

subgraph
    C1[] --> C2[]
    C2 --> C3[\n]
    C3 --> C4[]
end

```

> 面试官：垃圾回收算法有哪些？

>

> 候选人：主要有四种垃圾回收算法：

>

>

1. 标记-清除算法：分为标记和清除两个阶段，先标记需要回收的对象，然后统一回收。缺点是效率低且会产生内存碎片

>

>

2. 复制算法：将内存分成两块，每次只使用其中一块，GC时把存活对象复制到另一块，然后清空之前那块。优点是没有碎片，但浪费了一半内存。适合存活对象少的场景（如新生代）

>

>

3. 标记-整理算法：标记后不是直接清除，而是将存活对象向一端移动，然后清理边界外的内存。解决了碎片问题，但移动对象有开销

>

>

4. 分代收集算法：根据对象生存周期不同划分到不同区域，采用不同算法。新生代用复制算法，老年代用标记-整理算法

4.2 判断垃圾的方法 ★★★★★（高频）

> 面试官：判断垃圾的方法有哪些？

>

> 候选人：主要有两种判断垃圾的算法：

>

>

1. 引用计数法：为每个对象分配一个引用计数器，每有一个地方引用它时计数器+1，引用失效时-1。当计数器为0时表示对象可回收。缺点是不能解决循环引用问题

>

>

2. 可达性分析算法（根可达算法）：从GC Roots（如虚拟机栈中引用的对象、方法区中类静态属性引用的对象、本地方法栈中JNI引用的对象等）出发，向下追溯它们引用的对象。如果一个对象到GC Roots没有任何引用链相连，则标记为可回收对象。Java主要采用此算法

4.3 Minor GC vs Full GC ★★☆☆☆

> 面试官：MinorGC、MajorGC、FullGC的区别，什么场景触发Full GC？

>

> 候选人：

> - Minor GC：只针对年轻代进行回收，包括Eden区和Survivor区。触发条件是Eden区空间不足

> - Major GC：主要针对老年代进行回收，频率较低

>

GC：对整个堆内存（包括年轻代、老年代和元空间）进行回收。触发条件包括：System.gc()调用、老年代空间不足、空间分配担保失败、元空间不足等

4.4 哪些是GC Roots ★★★★★☆

5. 垃圾收集器

5.1 收集器关系图 ★★★★★☆

```

flowchart TB
    subgraph TB
        E[Serial\n] --> P[ParNew\n]
        P --> PS[Parallel Scavenge\n]
    end

    subgraph TB
        S[Serial Old\n] --> CMS[CMS\n]
        CMS --> M[Mixed GC]
        PS --> G1[G1\n]
    end

    G1 --> Z[ZGC\n]
    G1 --> SH[Shenandoah]
  
```

5.2 各收集器对比 ★★★★★☆

5.3 CMS收集器原理 ★★★★★☆

```

flowchart TD
    A["A[\nSTW]"] --> B["B[]"]
    B --> C["C[\nSTW]"]
    C --> D["D[]"]
    D --> E["E[]"]

    A --> A1["A1[GC Roots\n]"]
    C --> C1["C1[\n]"]
  
```

CMS四阶段：

> 面试官：CMS和G1的区别？

>

> 候选人：

- > 1. 使用范围不同：CMS是老年代收集器，配合新生代Serial/ParNew使用；G1收集范围是整个堆
- > 2. STW时间：CMS以最小停顿时间为目标；G1可建立可预测的停顿时间模型
- > 3. 碎片问题：CMS使用标记-清除算法会产生内存碎片；G1使用标记-整理，没有碎片
- > 4. 浮动垃圾：CMS会产生浮动垃圾，当浮动垃圾过多时会退化为Serial Old；G1没有浮动垃圾

5.4 G1收集器原理 ★★★★★☆（高频）

```

flowchart TB
    subgraph G1
        R1[Region 1\nEden]
        R2[Region 2\nSurvivor]
        R3[Region 3\nOld]
        R4[Region 4\nHumongous]
        R5[Region N]
    end

    subgraph GC
        A[GC] --> B[GC]
        B --> C[Full GC\n]
    end
  
```

G1特点：

- 区域化内存布局，整个堆分成多个Region
- 可预测停顿时间，通过设置期望停顿时间

- 优先回收价值最大的Region（垃圾多、存活对象少）
- 采用标记-整理算法，无内存碎片

G1回收过程：

1. ****年轻代GC****：Eden区满后进行STW清理，复制存活对象到Survivor区
2. ****混合GC****：老年代占比达到阈值后，进行混合回收
3. ****Full GC****：如果回收速度跟不上，产生Full GC

5.5 ZGC详解 ★★★☆☆（高频）

>

ZGC核心思想：通过着色指针和读屏障技术，在并发标记和并发移动对象时，保证应用线程访问到的对象总是有效的。

5.6 各种GC算法哪些阶段会STW ★★★★★☆

> 面试官：垃圾回收算法哪些阶段会stop the world?

>

>

候选人：以G1为例，标记-复制算法可以分为三个阶段：标记阶段、转移阶段、重定位阶段，其中标记阶段和转移阶段是STW的。

>

> G1停顿时间的瓶颈主要是标记-复制中的转移阶段，因为需要复制对象并更新引用。

>

> CMS的STW主要发生在初始标记和重新标记阶段。

6. 类加载机制

6.1 类加载流程 ★★★★★（高频）

flowchart TD

```

A[Loading] --> B[Verification]
B --> C[Preparation]
C --> D[Resolution]
D --> E[Initialization]
E --> F[Using]
F --> G[Unloading]

```

```

style A fill:#90EE90
style E fill:#FFB6C1

```

> 面试官：讲一下类加载过程？

>

> 候选人：类从被加载到虚拟机内存开始，到卸载出内存为止，整个生命周期包括：

>

>

1.

加载：通过类的全限定名获取.class文件的二进制字节流，将静态存储结构转化为方法区运行时的数据结构，在内存中生成Class对象

>

> 2. 链接：包括验证、准备、解析三个阶段

> - 验证：确保Class文件字节流符合要求

> - 准备：为静态变量分配内存并设置初始值

> - 解析：将符号引用替换为直接引用

>

> 3. 初始化：执行类的构造器方法()，这是编译器自动生成的

>

> 4. 使用和卸载：类被使用或卸载

6.2 双亲委派模型 ★★★★★（高频）

flowchart TB

```

A[Bootstrap\nClassLoader] --> B[Extension\nClassLoader]
B --> C[Application\nAppClassLoader]
C --> D[\nClassLoader]

D -->| | E{}
E -->| ? | F[]
E -->| | G[]

A -. ->| | B -. ->| | C

```

类加载器优先级（从高到低）：

6.3 双亲委派模型的作用 ★★★★★（高频）

> 面试官：Java中双亲委派是什么？有啥用？

>

>

候选人：双亲委派是指「一个类加载器要加载类时，先让父加载器去尝试加载，只有父加载器加载不了，自己才会去加载」。

>

> 核心作用：

> 1. 保证类的唯一性和安全性：避免同一个类被不同加载器重复加载，确保核心类不会被篡改

> 2. 实现类的复用：核心类只需要被顶层加载器加载一次，所有子加载器都能共享

>

>

举例：如果我们自己写了一个java.lang.String类，AppClassLoader会委派给Extension ClassLoader，再委派给Bootstrap ClassLoader。Bootstrap ClassLoader发现已经加载过JDK的String类，就直接返回，不会加载我们自定义的String类。

6.4 类加载器有哪些 ★★★★★☆

6.5 为什么要自定义ClassLoader ★★☆☆☆

6.6 SPI机制：打破双亲委派的典型场景 ★★★★★

6.6.1 什么是SPI？

SPI (Service Provider Interface) 是Java提供了一种服务发现机制，用于实现模块化和解耦。

```

flowchart LR
    subgraph SPI
        A[Service] --> B[ServiceLoader]
        B --> C[A]
        B --> D[B]
        B --> E[C]
    end

    subgraph
        F[META-INF/services]
    end

    C --> F
    D --> F
    E --> F

```

SPI的核心思想：接口由JDK/框架提供，实现由第三方厂商提供，运行时动态加载。

6.6.2 为什么需要SPI？

常见SPI应用场景：

6.6.3 JDBC SPI加载过程（经典案例）

```

// JDK - BootstrapClassLoader
java.sql.Driver

// - AppClassLoader
com.mysql.cj.jdbc.Driver
org.postgresql.Driver

```

问题：Driver接口由BootstrapClassLoader加载（因为在java.sql包下），但MySQL驱动是AppClassLoader加载

的。子类加载器无法使用父加载器加载的类，这违反了双亲委派！

flowchart TB

```
A[Bootstrap\nClassLoader] -->| | B[java.sql.Driver\n]
C[App\nClassLoader] -->| | D[com.mysql.cj.jdbc.Driver\n]
```

```
D -. ->| | B
```

6.6.4 SPI打破双亲委派的原理

解决方案：线程上下文类加载器（Thread Context ClassLoader）

```
//
ClassLoader cl = Thread.currentThread().getContextClassLoader();

// ServiceLoader
ServiceLoader<Driver> loader = ServiceLoader.load(Driver.class, cl);
```

```
sequenceDiagram
    participant T as
    participant S as ServiceLoader
    participant I as Driver
    participant M as MySQL

    T->>S: ServiceLoader.load(Driver.class)
    S->>T: Thread.currentThread().getContextClassLoader()
    T->>S: AppClassLoader
    S->>M: AppClassLoadercom.mysql.cj.jdbc.Driver
    M->>I: Driver
    I->>M:
```

关键代码：

```
// ServiceLoader.load()
public static <S> ServiceLoader<S> load(Class<S> service, ClassLoader loader) {
    // loadernull
    if (loader == null) {
        loader = ClassLoader.getSystemClassLoader();
    }
    // ServiceLoaderloader
    return new ServiceLoader<>(service, loader);
}

//
// com.mysql.cj.jdbc.Driver
public class Driver implements java.sql.Driver {
    static {
        // DriverManager
        DriverManager.registerDriver(new Driver());
    }
}
```

6.6.5 SPI实现示例

1. 定义服务接口（接口方）

```
// com.example.CacheService
package com.example;

public interface CacheService {
    void set(String key, String value);
    String get(String key);
    void delete(String key);
}
```

2. 实现服务（实现方）

```
// com.example.impl.RedisCache
package com.example.impl;

public class RedisCache implements CacheService {
    @Override
    public void set(String key, String value) {
        // Redis
    }

    @Override
    public String get(String key) {
        // Redis
    }
}
```

```

        return null;
    }

    @Override
    public void delete(String key) {
        // Redis
    }
}

```

3. 配置SPI文件（实现方）

```

# META-INF/services/com.example.CacheService
#

com.example.impl.RedisCache
com.example.impl.MemcacheCache

```

4. 加载服务（使用方）

```

public class CacheFactory {

    public static CacheService getCacheService() {
        // SPI
        ServiceLoader<CacheService> loader =
            ServiceLoader.load(CacheService.class);

        //
        for (CacheService cache : loader) {
            return cache; //
        }

        throw new RuntimeException("CacheService");
    }
}

```

6.6.6 SPI与双亲委派的关系总结

```

flowchart TB
    A[] --> B[]
    A --> C[]

    D[SPI] --> E[]
    D --> F[]

    G[] --> H[JDBC]
    G --> I[JNDI]
    G --> J[XML]

```

6.6.7 面试回答要点

> 面试官：什么场景会打破双亲委派？

> 候选人：主要有以下场景：

> 1. SPI机制：这是最典型的场景。比如JDBC驱动加载，java.sql.Driver接口由BootstrapClassLoader加载，但MySQL驱动com.mysql.cj.jdbc.Driver由AppClassLoader加载。驱动需要向DriverManager注册，但DriverManager是由BootstrapClassLoader加载的，它无法直接使用AppClassLoader加载的驱动。所以引入了线程上下文类加载器，让AppClassLoader可以逆向委托给BootstrapClassLoader加载的接口。

> 2. 热部署/OSGI：应用运行时动态替换类

> 3. Tomcat容器：每个Web应用有自己的ClassLoader，隔离Web应用之间的类

> 核心解决思路：通过Thread.currentThread().getContextClassLoader()获取当前线程的类加载器，这个类加载器默认是AppClassLoader，用它来加载SPI实现类，实现接口与实现的解耦。

7. 运行时常量池

7.1 三种常量池对比 ★★★☆☆

```

flowchart LR
    subgraph StringTable

```

```

    A["\hello\""]
end

subgraph Class
    B["\n#1 #2"]
end

subgraph
    C["\n"]
end

B -->|| C
A -->|intern| C

```

7.2 String.intern() ★★★★★☆

```

// JDK6:
// JDK7+:

String s1 = new String("hello");
s1.intern();
String s2 = "hello";
s1 == s2; // JDK6: false  JDK7+: true

```

> 面试官：String s = new String("abc") 执行过程中分别对应哪些内存区域？

>

> 候选人：

> - new指令创建的字符串对象是在堆内存上

> - "abc"是字符串常量，如果不存在，则在堆中创建字符串对象并加入字符串常量池

> - 所以如果abc不存在，会创建两个对象；存在则只创建一个对象

8. 字节码执行

8.1 解释执行 vs JIT编译 ★★★★★☆☆

```

flowchart LR
    A[] --> B[]
    B --> C[]

    D[] --> E[JIT]
    E --> F[]
    F --> G[]

    style E fill:#FFB6C1

```

8.2 热点代码检测 ★★★★★☆☆

9. 调优参数与工具

9.1 常用调优参数 ★★★★★☆☆

9.2 问题排查工具 ★★★★★☆☆

10. 高频面试题

10.1 内存与结构相关 ★★★★★★

> 面试官：JVM内存结构有几种内存溢出的情况？

>

> 候选人：

> 1. 堆内存溢出：java.lang.OutOfMemoryError: Java heap space，原因是创建大对象或内存泄露

> 2. 栈溢出：StackOverflowError（递归过深）或OutOfMemoryError（栈扩展失败）

> 3. 元空间溢出：java.lang.OutOfMemoryError: Metaspace，原因是类信息过多

> 4. 直接内存溢出：java.lang.OutOfMemoryError: Direct buffer memory

> 面试官：遇到过堆溢出的情况吗？如何解决？

>

> 候选人：堆溢出通常发生在程序持续创建对象且无法被GC及时回收的场景。

>

> 解决步骤：

> 1. 通过JVM参数-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=./heapdump.hprof捕获内存快照

> 2. 使用MAT或JProfiler分析快照，找到内存占用大的对象

> 3. 如果是内存泄漏：找到无用但未被回收的引用链，清理静态集合等

> 4. 如果是内存不足：调整堆大小-Xms2g -Xmx4g或优化代码

> 面试官：遇到过栈溢出的情况吗？如何解决？

>

> 候选人：栈溢出主要发生在递归调用没有正确终止条件时。

>

> 解决方式：

> 1. 检查递归逻辑，添加正确的终止条件

> 2. 将递归改写为迭代（如用循环替代）

> 3. 通过-Xss256k增大栈内存（但会影响线程数量）

> 4. 优化方法栈帧，减少局部变量

10.2 垃圾回收相关 ★★★★★

> 面试官：什么是Java里的垃圾回收？如何触发垃圾回收？

>

> 候选人：垃圾回收（GC）是自动管理内存的机制，负责自动释放不再被引用的对象所占用的内存。

>

> 触发方式：

> - 内存不足时自动触发

> - 调用System.gc()（只是建议，不保证立即执行）

> - JVM参数调整GC行为

> - 达到垃圾收集器内部阈值

> 面试官：垃圾回收器CMS和G1的区别？

>

> 候选人：

> 1. 使用范围：CMS是老年代收集器；G1是整堆收集器

> 2. 停顿时间：CMS追求最短停顿；G1可建立可预测停顿模型

> 3. 内存碎片：CMS用标记-清除有碎片；G1用标记-整理无碎片

> 4. 浮动垃圾：CMS会产生浮动垃圾；G1不会

> 面试官：什么情况下使用CMS，什么情况使用G1？

>

> 候选人：

> - CMS适用场景：对停顿时间要求敏感、老年代收集、可以接受一定内存碎片

> - G1适用场景：大堆内存（6GB以上）、需要可预测停顿时间、追求整体吞吐量

10.3 类加载相关 ★★★★★

> 面试官：讲一下类的加载和双亲委派原则

>

> 候选人：

> 类加载过程：加载→链接→初始化

> - 加载：将字节码读取到JVM

> - 链接：验证（安全检查）+ 准备（分配内存+初始值）+ 解析（符号引用转直接引用）

> - 初始化：执行静态代码块和静态变量赋值

>

>

双亲委派原则：类加载器收到加载请求时，先委派给父加载器处理，只有父加载器无法完成时，自己才尝试加载。

>

> 好处：保证类唯一性和安全性，比如自己写的String类不会被加载，因为会被委派给Bootstrap ClassLoader加载JDK的String

10.4 内存泄漏与溢出 ★★★★★☆

> 面试官：内存泄漏和内存溢出的理解？

>

> 候选人：

> - 内存泄漏：对象不再使用但仍被引用，导致GC无法回收，持续占用内存

> - 内存溢出：JVM无法申请到足够内存，抛出OutOfMemoryError

>

> 常见内存泄漏原因：

> - 静态集合未清理

> - 未关闭的资源（IO、连接）

> - ThreadLocal未remove

> 面试官：弱引用了解吗？举例说明在哪里可以用？

>

> 候选人：Java中的弱引用有四种类型：

> - 强引用：A a = new A()，永远不会GC

> - 软引用：SoftReference，内存不足时GC

> - 弱引用：WeakReference，下次GC一定回收

> - 虚引用：PhantomReference，必须配合ReferenceQueue，GC时收到通知

>

> 使用场景：WeakHashMap、ThreadLocal的Entry使用弱引用、缓存系统

10.5 对象创建过程 ★★★★★☆

> 面试官：创建对象的过程？

>

> 候选人：对象创建过程：

> 1. 类加载检查：检查常量池是否有类的符号引用，未加载则先加载

> 2. 分配内存：确定大小，分配内存（指针碰撞或空闲列表）

> 3. 初始化零值：保证字段可以不赋初值就使用

> 4. 设置对象头：哈希码、GC分代年龄、锁状态等

> 5. 执行init：执行构造方法，真正初始化对象

附录：面试高频问题速查

>

笔记说明：本笔记根据黑马程序员JVM面试课程和小林coding面试题整理，涵盖JVM核心面试知识点。使用mermaid图解便于理解，建议面试前快速回顾。